Agdasubst: Automated Reasoning for Unified Renaming and Substitution over Scoped and Multi-Sorted Syntax

MARIUS WEIDNER*, University of Freiburg, Germany

Mechanizing the metatheory of programming languages in proof assistants poses recurring challenges for variable binding and substitution. Every formalization requires proving standard lemmas about substitutions and applying them correctly. Explicit substitution calculi, for instance the σ-calculus, provide syntactic reduction rules that ensure confluence and strong normalization, allowing equations between terms to be compared via normal forms and giving an automatic procedure that eliminates manual lemma application.

Directly embedding explicit substitution laws as reduction rules in Agda using its rewrite mechanism is challenging due to the need for precise control over which definitions reduce and which remain symbolic, the dependence of substitutions on renamings for structurally recursive definitions, and the requirement to support multiple variable sorts. We address the latter by adapting the σ-calculus to intrinsically scoped, multi-sorted syntax, first handling renamings and substitutions separately, then presenting a uniform treatment that allows results about renamings and substitutions to be proven once for both, avoiding duplication.

1 Introduction

Mechanizing the metatheory of programming languages within interactive theorem provers often presents recurring uninteresting technical challenge, namely the handeling of variable binding and substitution. While such details are usually ignored in pen-and-paper proofs, their mechanization in proof assistants requires precise definitions and a significant amount of manual effort.

Historically, substitution has been treated as an meta-level operation. To formalize this, explicit substitution calculi [1] were introduced to bring substitutions down to a syntactic level. These calculi define substitution behavior through a set of reduction rules. With the σ-calculus, a terminating, confluent [10] and complete [19] rewrite system for λ-calculus terms with substitution has been discovered. Repeatedly applying the rules left-to-right yields a unique normal form, which can then be syntactically compared for equality.

The σ-calculus laws generalize to custom syntaxes without much difficulty [20], and it is tempting to assume that this would also make it easy to embed these laws into proof assistants and then solve equations between terms with substitution automatically by applying them from left to right, either via tactics or embedding as native reduction. In practice, however, when we try to embed the σ-calculus into Agda [17], two major challenges remain.

First, we need to distinguish between renamings, that is substitutions mapping variables to variables, and general substitutions, which map variables to terms, in order to obtain structurally recursive definitions [2]. In fact, this separation is useful in any way, there are situations where we want to reason about renamings alone. However, the separation leads to code duplication, since lemmas such as substitution preserves typing must be proven twice, once for the special case of renamings, and once again for substitutions, relying on the result about renamings.

*I hereby declare that I am the sole author and composer of my thesis and that no other sources or learning aids, other than those listed, have been used. Furthermore, I declare that I have acknowledged the work of others by providing detailed references of said work.
I hereby also declare that my thesis has not been prepared for another examination or assignment, either wholly or excerpts thereof.

Freiburg, XX.XX.2025	yadda yadda
Place, Date	Signature

Author's Contact Information: Marius Weidner, weidner@cs.uni-freiburg.de, University of Freiburg, Germany.

Second, in languages with multiple variable sorts, the combination of several syntactic categories and the separation of renamings and substitutions can lead to a combinatorial explosion of near-identical substitution definitions and lemmas.

When choosing a syntax representation in Agda, several options are available: extrinsically scoped and typed, intrinsically scoped but extrinsically typed, or intrinsically scoped and typed. We argue that the intrinsically scoped but extrinsically typed approach is the sweatspot between these extremes. In Agda, using a multi-sorted, intrinsically scoped syntax helps to avoid the combinatorial explosion of lemmas that arises from having multiple variable sorts, since we can define parallel, multi-sorted renaming and substitution uniformly across them. As we will see, adapting the σ-calculus to such a setting with multi-sorted, scoped syntax and first-class renamings is not particularly difficult when done carefully, yet it already yields significant benefits.

The remaining challenge caused by the separation of renamings and substitutions is more difficult to address. We build on the notion of Kits [4, 5, 16], a structure that captures the behaviour common to variables and terms, originally in an intrisnically typed setting. From Kits, one can define a uniform structure for renamings and substitutions, called Maps. This idea was later extended to multi-sorted and scoped syntax [18]. Our work follows the latter developments. We adapt the σ-calculus to a setting with Kits and Maps over multi-sorted and scoped syntax, treating renamings and substitutions uniformly.

Structure

The rest of this paper is structured in the following way:

- Section 2 introduces the theory and tools used in this work;
- Section 3 shows how to embed a scoped and multi-sorted version of the σ-calculus in Agda, treating renamings and substitutions separately;
- Section 4 explains the internals of the Agdasubst library, where renamings and substitutions can be treated uniformly;
- Section 5 shows how to use the Agdasubst library and includes case studies;
- Section 6 compares the Agdasubst library to similar works;
- Section 7 outlines possible future improvements to the Agdasubst library.

The supplement contains the complete Agda code underlying this paper.

2 Preliminaries

This section introduces the concepts and tools required for the developments in this work. We summarize the following components:

- Lesser known Agda features: instance arguments, opaque blocks, and rewrite rules;
- The σ-calculus;
- Multi-sorted and scoped syntaxes in Agda.

2.1 Agda

Agda is a dependently typed programming language and proof assistant based on Martin-Löf type theory [15]. It allows users to write programs and proofs in a functional language, verifying correctness through type checking.

In addition to its well-known features such as indexed and parameterized inductive data types, dependent (co)pattern matching [8], implicit arguments and records, Agda supports three less commonly used but important features relevant to this work.

Instance Arguments

Instance arguments, written using double braces {{...}}, differ from implicit arguments {...} in that they are automatically resolved via an instance search algorithm, rather than the more simple implicit argument resolution limited to the current scope. When a function expects an instance argument, Agda attempts to infer and insert it from values marked with the instance keyword. This mechanism enables type-directed programming patterns similar to type classes in Haskell [25]. As an example, consider the following definition of a record for types with a default value:

```
record Default {ℓ} (A : Set ℓ) : Set ℓ where
  field default : A
```

The record can be opened using instance argument resolution for its fields:

```
open Default {{...}}
```

We can now declare an instance for natural numbers:

```
instance
  default-ℕ : Default ℕ
  default-ℕ .default = 0
```

This enables implicit usage of the default value via instance search:

```
_ : default ≡ 0
_ = refl
```

Here, Agda infers from the type ℕ of 0 that the instance default-ℕ applies for default, inserting it automatically to conclude the equation holds definitionally.

Opaque Blocks

Agda supports opaque definitions [13], which allow certain parts of the code to be treated as non-unfolding symbols, similar to abstract definitions. This feature is useful for controlling definitional equality and can be used to prevent specific definitions from being unfolded during type or conversion checking.

A block is marked as opaque by preceding it with the opaque keyword. All definitions within the block become opaque, meaning they are excluded from definitional equality checks. However, their names and types remain visible and can still be used throughout the code. As an example consider:

```
opaque
  forty-two : ℕ
  forty-two = 42
```

In this example, forty-two is defined as the numeral 42, but it is opaque outside the block. This means that although forty-two still has type ℕ and can be used in expressions and proofs, it does not reduce to 42 outside the block during definitional equality checks:

```
_ : forty-two ≡ 42
_ = {! !} -- not provable.
```

In subsequent opaque blocks, the type checker can be instructed to selectively unfold previously opaque definitions using the unfolding keyword. Unfolding the definition will reduce the definition to 42 again, as expected:

```
opaque
  unfolding forty-two

  _ : forty-two ≡ 42
  _ = refl
```

Rewrite Rules

In the context of this work, the most import feature of Agda are user defined rewrite rules [7, 9] that extend the system's notion of definitional equality by integrating them directly into type checking. A function definition in Agda qualifies to be added as rewrite rule, if it is of the form

$$\mathsf{eq} : (x_1 : A_1) \ \ldots \ (x_k : A_k) \to \mathsf{f}\, p_1 \ \ldots \ p_n = v$$

and satisfies the following three conditions:

(1) The head symbol f must be a postulate, a defined function, or a constructor applied to fully general parameters, i.e., the parameters must be distinct variables.
(2) Each variable $x_1, \ldots, x_k$ must occur at least once in a pattern position among $p_1, \ldots, p_n$[1].
(3) The left side $\mathsf{f}\, p_1 \ \ldots \ p_n$ must be neutral, i.e., it should not reduce any further under existing definitions.

As an example, consider the right identity law for addition:

```
+-id_r : ∀ n → n + 0 ≡ n
+-id_r zero    = refl
+-id_r (suc n) = cong suc (+-id_r n)
```

The type of the lemma satisfies the conditions above and can be registered as a rewrite rule using the REWRITE pragma:

```
{-# REWRITE +-id_r #-}
```

From this point forward, Agda treats $n + 0$ as definitionally equal to n:

```
_ : ∀ {n} → n + 0 ≡ n
_ = refl
```

To ensure consistency, Agda requires that all rewrite rules satisfy the three conditions stated above, and that the combined set of rules is both terminating and confluent. However, termination is not currently checked by Agda, it is only observable indirectly when type checking fails to terminate. Confluence can be checked using the `--local-confluence-check` and `--confluence-check` flags, but both checks are necessarily conservative.

Symbols

Variables x

innermost variable	zero
outer variables	suc x

Terms t

variable usage	`x
abstraction	$\lambda.\, t$
application	$t_1\ t_2$
substitution lookup	x & σ
substitution application	$t \cdots \sigma$

Substitutions σ

identity	id
weakening	wk
extension	$t \cdot \sigma$
composition	σ_1 ; σ_2

Reduction

Definitional Laws

id	x & id	= `x
wk	x & wk	= `(suc x)
ext$_Z$	zero & $(t \cdot \sigma)$	= t
ext$_S$	suc x & $(t \cdot \sigma)$	= x & σ
comp	x & $(\sigma_1 ; \sigma_2)$	= $(x$ & $\sigma_1) \cdots \sigma_2$

Traversal Laws

var	(`x) $\cdots \sigma$	= x & σ
lam	$(\lambda.\, t) \cdots \sigma$	= $\lambda.(t \cdots (\uparrow \sigma))$
app	$(t_1\ t_2) \cdots \sigma$	= $(t_1 \cdots \sigma)\ (t_2 \cdots \sigma)$

Interaction Laws

comp-id$_L$	id ; σ	= σ
comp-id$_R$	σ ; id	= σ
associativity	$(\sigma_1 ; \sigma_2) ; \sigma_3$	= $\sigma_1 ; (\sigma_2 ; \sigma_3)$
distributivity	$(t \cdot \sigma_1) ; \sigma_2$	= $(t \cdots \sigma_2) \cdot (\sigma_1 ; \sigma_2)$
interact	wk ; $(t \cdot \sigma)$	= σ
η-id	(`zero)$\cdot$wk	= id
η-law	(zero & σ)$\cdot$(wk ; σ)	= σ

Monad Laws

right-id	$t \cdots$ id	= t
compositionality	$(t \cdots \sigma_1) \cdots \sigma_2$	= $t \cdots (\sigma_1 ; \sigma_2)$

Fig. 1. The three-sorted σ-calculus.

2.2 The Sigma Calculus

Explicit substitution calculi were introduced to formalize the traditionally meta-theoretic treatment of substitutions. Unlike the standard approach, where substitution is defined at the meta-level, these calculi treat substitution primitives as syntactic constructs and define their behavior by reduction rules. Several explicit substitution systems have been proposed. The σ-calculus [1], the first of its kind, introduces substitution primitives such as identity, extension, shifting, and composition. The $\sigma^{\Uparrow}$-calculus [10] extends this with a primitive for lifting.

The σ_{SP}-calculus is of particular interest because it includes additional rules to ensure confluence. It is sound, confluent, and complete [20]. As a result, any valid equality involving substitutions can be derived using its rules alone. In the following, we present our own three-sorted version of the σ-calculus, using the λ-calculus as the object language in Figure 1. Our system closely follows the σ_{SP}-calculus.

The syntax is sorted into de Bruijn variables, terms, and substitution primitives. We use de Bruijn variables to obtain α-equivalence for free. The expression $\uparrow\sigma$ is syntactic sugar for (`zero)$\cdot$(σ ; wk). Unlike most presentations in the

[1]See https://agda.readthedocs.io/en/latest/language/rewriting.html for a precise definition of pattern positions.

```
data Kind : Set where
  ★ : Kind

data Type (n : ℕ) : Set where
  ‘_        : Fin n → Type n
  ∀[α:_]_ : Type (suc n) → Type n
  _⇒_       : Type n → Type n → Type n

data Expr (n m : ℕ) : Set where
  ‘_    : Fin m → Expr n m
  λx_   : Expr n (suc m) → Expr n m
  Λα_   : Expr (suc n) m → Expr n m
  _·_   : Expr n m → Expr n m → Expr n m
  _•_   : Expr n m → Type n → Expr n m
```

```
data Sort : Set where
  expr type kind : Sort

Scope = List Sort
Scoped = Scope → Sort → Set

data _∋_ : Scoped where
  zero : ∀ {S s} → (s :: S) ∋ s
  suc  : ∀ {S s s′} → S ∋ s → (s′ :: S) ∋ s

data _⊢_ : Scoped where
  ‘_        : S ∋ s → S ⊢ s
  λx_       : (expr :: S) ⊢ expr → S ⊢ expr
  Λα_       : (type :: S) ⊢ expr → S ⊢ expr
  ∀[α:_]_ : S ⊢ kind → (type :: S) ⊢ type → S ⊢ type
  _·_       : S ⊢ expr → S ⊢ expr → S ⊢ expr
  _•_       : S ⊢ expr → S ⊢ type → S ⊢ expr
  _⇒_       : S ⊢ type → S ⊢ type → S ⊢ type
  ★         : S ⊢ kind
```

Fig. 2. Classically Scoped Syntax vs. Multi-Sorted and Scoped Syntax

literature, we include explicit syntax for variable usage within terms. This requires distinguishing between suc and wk, depending on whether they appear in the sort of variables or substitutions. The reduction rules are grouped into four categories.

- **Definitional Laws** describe how variables are looked up in substitutions;
- **Interaction Laws** express equalities between substitution expressions;
- **Traversal Laws** describe how substitutions propagate through term syntax and under binders;
- **Monad Laws** describe the behavior of substitutions when applied to terms.

We do not include reduction rules for the λ-calculus itself, such as β-reduction, as done for example in the $\lambda\sigma$-calculus [10]. Our focus is not on the reduction of the object language, but on the behavior of substitutions applied to terms. The reduction strategy of the object language is part of its definition and is left unspecified. The only rules involving the syntax that we require are those traversing terms, when a substitution is applied.

We believe the three-sorted σ-calculus is terminating and confluent. Although we do not give a formal proof, it is a reasonable variant of the σ_{SP}-calculus. Renamings, needed to define substitutions structurally in Agda, are believed to break completeness [23]. For this reason, we do not consider completeness a central property in this work. Still, we try to keep the space of solvable equations as large as possible.

2.3 Mutli-Sorted and Scoped Syntaxes

Unlike the simply typed λ-calculus, many formal languages have more than one sort of syntax. For example, System F [11] has kinds, types and expressions. Further, to avoid scoping related errors, we want syntaxes to enforce correct scoping intrinsically. This is usually done in Agda by a definition of the syntax as a family of types indexed by the set of its free variables.

On the left side of Figure 2, we start with a standard intrinsically scoped syntax for System F. Types are indexed by the number of free type variables, and expressions are indexed by both the number of free type and expression variables. Variables are represented using de Bruijn indices of type Fin n, that is, as natural numbers strictly less than the number of free variables. This guarantees that only well scoped terms can be formed.

This approach is simple but has drawbacks. Types and expressions are separate datatypes, so we would need to define multiple substitution operations: type-in-type, expr-in-expr, and type-in-expr. These substitutions need to be handled and related manually, which adds boilerplate.

To avoid this, we use a multi-sorted syntax. Terms form a single datatype indexed by a list of variable sorts and a target sort. The type $S \vdash s$ describes a term of sort s with free variables of sorts in S. Variable positions are given by $S \ni s$, a proof that a variable of sort s appears in S at some position. This position acts as the de Bruijn index. This gives us well scoped and well sorted terms by construction. The right side of Figure 2 shows this version. With this unified syntax, we can define scoped and multi-sorted renamings and substitutions that work uniformly across sorts.

3 The Multi-Sorted and Scoped Sigma Calculus with First-Class Renamings

This section gives a direct embedding of the σ-calculus rules as native reduction rules in Agda using the rewrite mechanism for the muti-sorted and scoped version of System F from Figure 2 as an example for any custom syntax. When adapting the σ-calculus from Section 2.2 to custom multi-sorted and scoped syntaxes in Agda, we face the problem that, in Agda, substitutions for syntaxes with binders cannot be defined directly by structural recursion. To obtain well-founded definitions, we need to define renamings first, which are substitutions that only map to variables.muti-sorted and scoped version of System F from Section 2.3 as an example for any custom syntax.

When adapting the σ-calculus from Section 2.2 to custom multi-sorted and scoped syntaxes in Agda, we face the problem that, in Agda, substitutions for syntaxes with binders cannot be defined directly by structural recursion. To obtain well-founded definitions, we need to define renamings first, which are substitutions that only contain variables. After we defined renamings and substitutions, we revisit the σ-calculus and adapt its laws to this setting.

3.1 Scoped and Multi-Sorted Renamings and Substitutions

We define renamings and substitutions uniformly over all variable sorts by indexing them over scopes, as shown in Figure 3. A renaming from scope S_1 to S_2 maps each variable of a given sort in S_1 to a variable of the same sort in S_2. Similarly, a substitution maps variables to terms of the corresponding sort.

The order of definitions is chosen carefully. We first define the renaming type and its primitive operations: identity, extension and weakening. Substitutions are then defined in terms of renamings, using weakening to define lifting.

To support all composition combinations of renamings and substitutions, we provide four forward composition primitives: renaming composed with renaming, renaming with substitution[2], substitution with renaming, and substitution with substitution.

The traversal functions, $_\cdots^R_$ for renaming and $_\cdots^S_$ for substitution, are defined by structural induction on the syntax. Each clause applies a renaming or substitution to a object language term, lifting the renaming or substitution, when going under a binder. The clauses in both definitions correspond directly to the traversal laws of the σ-calculus, and therefore, we want these functions to reduce during type checking. When adapting this framework to arbitrary

[2]The first two composition operators can technically be defined as a single definition together with composition of two renamings, they are simply forward function composition, but we choose to keep them separated.

```
opaque
  _→ᴿ_ : Scope → Scope → Set
  S₁ →ᴿ S₂ = ∀ {s} → S₁ ∋ s → S₂ ∋ s

  _&ᴿ_ : S₁ ∋ s → S₁ →ᴿ S₂ → S₂ ∋ s
  x &ᴿ ρ = ρ x

  idᴿ : S →ᴿ S
  idᴿ x = x

  wk : S →ᴿ (s :: S)
  wk x = suc x

  _·ᴿ_ : S₂ ∋ s → S₁ →ᴿ S₂ → (s :: S₁) →ᴿ S₂
  (x ·ᴿ _) zero    = x
  (_ ·ᴿ ρ) (suc x) = ρ x

  _;ᴿᴿ_ : S₁ →ᴿ S₂ → S₂ →ᴿ S₃ → S₁ →ᴿ S₃
  (ρ₁ ;ᴿᴿ ρ₂) x = ρ₂ (ρ₁ x)

_↑ᴿ_ : S₁ →ᴿ S₂ → ∀ s → (s :: S₁) →ᴿ (s :: S₂)
ρ ↑ᴿ _ = zero ·ᴿ (ρ ;ᴿᴿ wk)

_···ᴿ_ : S₁ ⊢ s → S₁ →ᴿ S₂ → S₂ ⊢ s
-- Traversal Laws
(` x)          ···ᴿ ρ = ` (x &ᴿ ρ)
(λx e)         ···ᴿ ρ = λx (e ···ᴿ (ρ ↑ᴿ _))
(Λα e)         ···ᴿ ρ = Λα (e ···ᴿ (ρ ↑ᴿ _))
(∀[α: k ] t)   ···ᴿ ρ = ∀[α: k ···ᴿ ρ ] (t ···ᴿ (ρ ↑ᴿ _))
(e₁ · e₂)      ···ᴿ ρ = (e₁ ···ᴿ ρ) · (e₂ ···ᴿ ρ)
(e • t)        ···ᴿ ρ = (e ···ᴿ ρ) • (t ···ᴿ ρ)
(t₁ ⇒ t₂)      ···ᴿ ρ = (t₁ ···ᴿ ρ) ⇒ (t₂ ···ᴿ ρ)
★              ···ᴿ ρ = ★
```

```
opaque
  unfolding
    _→ᴿ_ _&ᴿ_ idᴿ wk _·ᴿ_ _;ᴿᴿ_

  _→ˢ_ : Scope → Scope → Set
  S₁ →ˢ S₂ = ∀ {s} → S₁ ∋ s → S₂ ⊢ s

  _&ˢ_ : S₁ ∋ s → S₁ →ˢ S₂ → S₂ ⊢ s
  x &ˢ σ = σ x

  idˢ : S →ˢ S
  idˢ = `_

  _·ˢ_ : S₂ ⊢ s → S₁ →ˢ S₂ → (s :: S₁) →ˢ S₂
  (t ·ˢ _) zero    = t
  (_ ·ˢ σ) (suc x) = σ x

  _;ᴿˢ_ : S₁ →ᴿ S₂ → S₂ →ˢ S₃ → S₁ →ˢ S₃
  (ρ₁ ;ᴿˢ σ₂) x = σ₂ (ρ₁ x)

  _;ˢᴿ_ : S₁ →ˢ S₂ → S₂ →ᴿ S₃ → S₁ →ˢ S₃
  (σ₁ ;ˢᴿ ρ₂) x = (σ₁ x) ···ᴿ ρ₂

_↑ˢ_ : S₁ →ˢ S₂ → ∀ s → (s :: S₁) →ˢ (s :: S₂)
(σ ↑ˢ _) = (` zero) ·ˢ (σ ;ˢᴿ wk)

_···ˢ_ : S₁ ⊢ s → S₁ →ˢ S₂ → S₂ ⊢ s
-- Traversal Laws
(` x)          ···ˢ σ = (x &ˢ σ)
(λx e)         ···ˢ σ = λx (e ···ˢ (σ ↑ˢ _))
(Λα e)         ···ˢ σ = Λα (e ···ˢ (σ ↑ˢ _))
(∀[α: k ] t)   ···ˢ σ = ∀[α: k ···ˢ σ ] (t ···ˢ (σ ↑ˢ _))
(e₁ · e₂)      ···ˢ σ = (e₁ ···ˢ σ) · (e₂ ···ˢ σ)
(e • t)        ···ˢ σ = (e ···ˢ σ) • (t ···ˢ σ)
(t₁ ⇒ t₂)      ···ˢ σ = (t₁ ···ˢ σ) ⇒ (t₂ ···ˢ σ)
★              ···ˢ σ = ★

opaque
  unfolding
    _→ᴿ_ _&ᴿ_ idᴿ wk _·ᴿ_ _;ᴿᴿ_
    _→ˢ_ _&ˢ_ idˢ _·ˢ_ _;ᴿˢ_ _;ˢᴿ_

  _;ˢˢ_ : S₁ →ˢ S₂ → S₂ →ˢ S₃ → S₁ →ˢ S₃
  (σ₁ ;ˢˢ σ₂) x = (σ₁ x) ···ˢ σ₂
```

Fig. 3. Scoped and Multi-Sorted Renamings and Substitutions

custom syntaxes, the traversal functions are the only definitions that must be provided by the user. All other definitions can be stated independently of the concrete syntax.

Renaming and substitution primitives are declared opaque, which blocks their reduction. This abstraction is essential for formulating the σ-calculus laws, where the primitives must act as uninterpreted symbols rather than as concrete functions. Blocking reduction in this way allows rewrite rules to apply predictably. The types for renamings and

```
-- Definitional Laws
id^R : x &^R id^R          ≡ x
wk : x &^R wk {s = s'} ≡ suc x
ext₀^R : zero &^R (x ·^R ρ)    ≡ x
ext_s^R : (suc x') &^R (x ·^R ρ) ≡ x' &^R ρ

id^S : x &^S id^S          ≡ ‘ x
ext₀^S : zero &^S (t ·^S σ)    ≡ t
ext_s^S : (suc x) &^S (t ·^S σ) ≡ x &^S σ

comp^RR : x &^R (ρ₁ ;^RR ρ₂) ≡ (x &^R ρ₁) &^R ρ₂
comp^RS : x &^S (ρ₁ ;^RS σ₂) ≡ (x &^R ρ₁) &^S σ₂
comp^SR : x &^S (σ₁ ;^SR ρ₂) ≡ (x &^S σ₁) ⋯^R ρ₂
comp^SS : x &^S (σ₁ ;^SS σ₂) ≡ (x &^S σ₁) ⋯^S σ₂

{-# REWRITE
  id^R wk ext₀^R ext_s^R
  id^S ext₀^S ext_s^S
  comp^RR comp^RS comp^SR comp^SS

  comp-id_l^RR comp-id_l^RS comp-id^RRR
  comp-id^RSS comp-id^RSR comp-id_l^SS
  associativity^RRR associativity^RRS
  associativity^RSR associativity^RSS
  associativity^SRR associativity^SRS
  associativity^SSR associativity^SSS
  distributivity^RR distributivity^RS
  distributivity^SR distributivity^SS
  interact^R interact^S
  η-id η-law^R η-law^S

  right-id^R right-id^S
  compositionality^RR compositionality^RS
  compositionality^SR compositionality^SS

  coincidence
  coincidence-fold^V coincidence-fold^T
#-}
```

```
-- Interaction Laws
comp-id_l^RR : id^R ;^RR ρ ≡ ρ; comp-id_l^RS : id^R ;^RS σ ≡ σ
comp-id^RRR : ρ ;^RR id^R ≡ ρ
comp-id^RSS : σ ;^SS id^S ≡ σ; comp-id^RSR : σ ;^SR id^R ≡ σ
comp-id_l^SS : id^S ;^SS σ ≡ σ

associativity^RRR : (ρ₁ ;^RR ρ₂) ;^RR ρ₃ ≡ ρ₁ ;^RR (ρ₂ ;^RR ρ₃)
associativity^RRS : (ρ₁ ;^RR ρ₂) ;^RS σ₃ ≡ ρ₁ ;^RS (ρ₂ ;^RS σ₃)
associativity^RSR : (ρ₁ ;^RS σ₂) ;^SR ρ₃ ≡ ρ₁ ;^RS (σ₂ ;^SR ρ₃)
associativity^RSS : (ρ₁ ;^RS σ₂) ;^SS σ₃ ≡ ρ₁ ;^RS (σ₂ ;^SS σ₃)
associativity^SRR : (σ₁ ;^SR ρ₂) ;^SR ρ₃ ≡ σ₁ ;^SR (ρ₂ ;^RR ρ₃)
associativity^SRS : (σ₁ ;^SR ρ₂) ;^SS σ₃ ≡ σ₁ ;^SS (ρ₂ ;^RS σ₃)
associativity^SSR : (σ₁ ;^SS σ₂) ;^SR ρ₃ ≡ σ₁ ;^SS (σ₂ ;^SR ρ₃)
associativity^SSS : (σ₁ ;^SS σ₂) ;^SS σ₃ ≡ σ₁ ;^SS (σ₂ ;^SS σ₃)

distributivity^RR : (x ·^R ρ₁) ;^RR ρ₂ ≡ ((x &^R ρ₂) ·^R (ρ₁ ;^RR ρ₂))
distributivity^RS : (x ·^R ρ₁) ;^RS σ₂ ≡ ((x &^S σ₂) ·^S (ρ₁ ;^RS σ₂))
distributivity^SR : (t ·^S σ₁) ;^SR ρ₂ ≡ ((t ⋯^R ρ₂) ·^S (σ₁ ;^SR ρ₂))
distributivity^SS : (t ·^S σ₁) ;^SS σ₂ ≡ ((t ⋯^S σ₂) ·^S (σ₁ ;^SS σ₂))

interact^R : wk ;^RR (x ·^R ρ) ≡ ρ; interact^S : wk ;^RS (t ·^S σ) ≡ σ

η-id : zero {S = S} {s = s} ·^R wk ≡ id^R
η-law^R : (zero &^R ρ) ·^R (wk ;^RR ρ) ≡ ρ
η-law^S : (zero &^S σ) ·^S (wk ;^RS σ) ≡ σ


-- Monad Laws
right-id^R : t ⋯^R id^R ≡ t
right-id^S : t ⋯^S id^S ≡ t

compositionality^RR : (t ⋯^R ρ₁) ⋯^R ρ₂ ≡ t ⋯^R (ρ₁ ;^RR ρ₂)
compositionality^RS : (t ⋯^R ρ₁) ⋯^S σ₂ ≡ t ⋯^S (ρ₁ ;^RS σ₂)
compositionality^SR : (t ⋯^S σ₁) ⋯^R ρ₂ ≡ t ⋯^S (σ₁ ;^SR ρ₂)
compositionality^SS : (t ⋯^S σ₁) ⋯^S σ₂ ≡ t ⋯^S (σ₁ ;^SS σ₂)

-- Coincidence Laws
coincidence : t ⋯^S (ρ ;^RS id^S) ≡ t ⋯^R ρ
coincidence-fold^V : t ⋯^S ((‘ x) ·^S (ρ ;^RS id^S)) ≡ t ⋯^R (x ·^R ρ)
coincidence-fold^T : t ⋯^S ((t' ⋯^R ρ) ·^S (ρ ;^RS id^S)) ≡ (t ⋯^S (t' ·^S id^S)) ⋯^R ρ
```

Fig. 4. The Scoped and Multi-Sorted σ-calculus with First-Class Renamings and Substitutions.

substitutions are also declared opaque to prevent misuse as actual functions. Their map-like behavior remains accessible through the _&_ operator.

3.2 Scoped and Multi-Sorted Sigma Calculus with First-Class Renamings and Substitutions

We now adapt the rules from Section 2.2 to the setting with renamings. One might restrict the relevance of renamings to the structural definition of substitutions and state the laws only for substitutions. However, it is often useful to reason about renamings directly, for instance in the context morphism lemma [12]. We therefore treat renamings as first-class constructs. Figure 4 presents the adapted σ-calculus laws. They all satisfy the requirements from Section 2.1 to serve as valid rewrite rules.

The definitional laws now involve primitives for both renamings and substitutions. The interaction laws likewise cover all operators. Many of these laws are similar in shape, e.g. we require eight associativity laws. Both definitional and interaction laws can be proven independently of the concrete syntax. In contrast, the monad laws must be shown by induction over the syntax and are thus syntax-specific. We highlight this by marking these laws in red throughout this work.

Stark et al. noticed that a further category emerges from the interaction between renamings and substitutions. The coincidence laws govern the case where substitutions are intertwinded with renamings, but the renamings can be separated from the substitutions. The coincidence and the coincidence-fold laws allow us to recover renamings from their embedding as substitutions via the identity substitution. Extracting arbitrary renamings from deeply nested substitutions would require a dedicated solving strategy, which we leave for future work, but consider important for a more complete account.

The proofs for the laws can be established by unfolding the definitions of the primitives for renamings and substitutions. In the implementation we make use of function extensionality in order to prove the σ-calculus laws. It is known that with a little more effort, this assumption is not necessary, as shown for instance by Saffrich [18].

3.3 Discussion

The laws presented in Section 3.2 are an extension of the classic σ-calculus rules to a setting with renamings. They fulfill the requirements for valid rewrite rules without further modification. This is a consequence of marking the relevant symbols opaque, inside proofs of the laws themselves, the definitions can still unfold when needed, while externally they behave as uninterpreted symbols and are reduced only by the σ-calculus laws.

When reasoning solely about renamings or substitutions, the laws suffice to solve all equations automatically. Only when crossing the boundary between renamings and substitutions, and thus relying on coincidence laws, additional manual effort might sometimes become necessary.

The main drawback of this approach is the unavoidable duplication of similar definitions and proofs. While associativity laws do not cause difficulties for the user, even though there are many of them, since they can be stated independently of the syntax, the situation is different for traversal, monad, and coincidence laws. These must be provided explicitly by the user and consist of two traversal definitions, two right-identity laws, and four compositionality laws. Although each of them is similar in shape, taken together they still contribute a noticeable amount of boilerplate. Even if we would be able to derive the syntax-dependent definitions and laws, we still would need to prove every later result about substitutions twice, first for the special case of renamings and then for substitutions.

4 The Agdasubst Library

This section introduces the Agdasubst library. It builds on the ideas from Section 3, but removes the strict separation between variables and terms by introducing a construct that can represent either, which we call Kits. Using Kits, we

can define renamings and substitutions uniformly through a single notion, which we call Maps. With Kits and Maps in place, we can unify the definitions and proofs from Section 3, which were previously similar in shape.

The idea of Kits originates from McBride [16], where it was first formulated in an intrinsically typed setting. It was later adapted by Saffrich [18] to a multi-sorted and scoped setting. We frist recall the relevant definitions from the latter in Sections 4.1 and 4.2, and then show how the σ-calculus laws can be adapted to the setting of Kits and Maps. We omit most implementation details of Kits and Maps, except for those directly relevant to the embedding of the σ-calculus laws. For a full account of Kits, Maps, and their extensions, we refer the reader to the work of Saffrich.

The library, and with it the whole of the next chapter, abstracts over a set of sorts *Sort* : Set, a scoped and multi-sorted syntax _⊢_ : List *Sort* → *Sort* → Set, and a variable constructor ‘_ : S ∋ s → S ⊢ s.

4.1 Kits

A Kit is a structure that abstracts over terms and variables. The intended usage is to instantiate this structure exactly twice, first for variables and then for terms. Definitions can be written uniformly, parameterized over a Kit, and are consequently applicable to both cases.

```
record Kit (k : Mode) : Set₁ where
  _∋/⊢ᴷ_ = image k
  field
    K-id/‘ : S ∋ s → S ∋/⊢ᴷ s
    K-‘/id : S ∋/⊢ᴷ s → S ⊢ s
    K-wk  : ∀ s′ → S ∋/⊢ᴷ s → (s′ ∷ S) ∋/⊢ᴷ s
```

To distinguish the only two instances of Kits on the type level, Kits are indexed over a boolean-like structure data Mode : Set where V^M T^M : Mode. The image function computes, given a Mode, whether the Kit corresponds to a variable or term:

```
image : Mode → Scoped
image Vᴹ = _∋_
image Tᴹ = _⊢_
```

The Kit record has multiple fields:

- K-id/‘ converts a variable into an element of S ∋/⊢^{K} s. For the variable Kit, _∋/⊢^{K}_ is instantiated to _∋_, so this operation acts as the identity. For the term Kit, it is instantiated to _⊢_, so this operation corresponds to the variable constructor.
- K-‘/id converts an element of S ∋/⊢^{K} s into a term S _⊢_ s, and is the dual to K-id/‘.
- K-wk shifts the de Bruijn indices in a variable or term. The newly introduced, unused variable zero can assume any sort s'. For variables, K-wk corresponds to the successor constructor suc. For terms, K-wk need to apply a weakening renaming to the term using traversal.

Because rewrite rules cannot mention record fields directly, we provide for each field a corresponding version without the K-prefix. These are declared as opaque functions, so they can be used as symbols in rewrite rules, while still unfolding to the actual fields when needed. Going forwards, we assume all opaque defintions unfolding any time throughout this section. It is only outside the library, when we have the σ-calculus laws in place, when we want

definitions to act as uninterpreted symbols. The Kit record further consists of axioms, that constrain the instantition, for example, that ∀ (x : S ∋ s) → K-‘/id (K-id/‘ x) ≡ ‘ x. We will skip over these axioms from now on, since for both variables and terms all of them hold almost entirely trivially.

We can now define the usual operations for renamings and substitutions. The definitions are included directly in the Kit record, so they are implicitly parameterized over a kit. All the primitves are marked opaque.

```
_→ᴷ_ : Scope → Scope → Set
S₁ →ᴷ S₂ = ∀ s → S₁ ∋ s → S₂ ∋/⊢ᴷ s

_&_ : ∀ {s} → S₁ ∋ s → S₁ →ᴷ S₂ → S₂ ∋/⊢ᴷ s
x & φ = φ _ x

id : S →ᴷ S
id s x = K-id/‘ x

_·_ : S₂ ∋/⊢ᴷ s → S₁ →ᴷ S₂ → (s :: S₁) →ᴷ S₂
(x/t · φ) _ zero    = x/t
(x/t · φ) _ (suc x) = φ _ x

_;wk : ∀ {s} → S₁ →ᴷ S₂ → S₁ →ᴷ (s :: S₂)
_;wk φ _ x = K-wk _ (φ _ x)

wk : S →ᴷ (s :: S)
wk = id ;wk
```

The type for Maps $S_1 \to^K S_2$ unifies renamings $S_1 \to^R S_2$ and substitutions $S_1 \to^S S_2$. For $M = \mathsf{V}^M$ the image function computes to the type of variables and thus the Map acts as renaming. Similarly, in the case that $K = \mathsf{T}^M$, the Map corresponds to a substitution. The identity map, lookup and extension operations behave analogously in both the renaming and substitution cases. Since term traversal is not yet available, composition cannot be defined at this stage. Instead, we introduce a generalized weakening operator _;wk, which can later be rewritten as a composition with renamings. Similarly, once term traversal is available, the id/‘ and ‘/id operators can be expressed entirely in terms of the primitive operations. We provide a shorthand for lifting a map:

```
_↑_ : S₁ →ᴷ S₂ → ∀ s → (s :: S₁) →ᴷ (s :: S₂)
φ ↑ s = (id/‘ zero) · (φ ;wk)
```

and for referring to a Kit explicitly in certain cases:

```
_∋/⊢[_]_ : Scope → Kit M → Sort → Set
S ∋/⊢[ K ] s = Kit._∋/⊢ᴷ_ K S s

_–[_]→_ : Scope → Kit M → Scope → Set
S₁ –[ K ]→ S₂ = Kit._→ᴷ_ K S₁ S₂

id[_] : (K : Kit M) → S –[ K ]→ S
id[ K ] = Kit.id K
```

To define the term Kit instances, we need to know how a Map is applied to a term. The user therefore has to provide a traversal function and proof that applying the identity map leaves a term unchanged:

```
_⋯_ : ∀ {{K : Kit M}} → S₁ ⊢ s → S₁ –[ K ]→ S₂ → S₂ ⊢ s
⋯-right-id : ∀ {{K : Kit M}} → (t : S ⊢ s) → t ⋯ id[ K ] ≡ t
```

Equipped with the traversal function we can define the two intended instances for Kits:

```
instance
  V : Kit V^M
  V = record
    { K-id/` = λ x → x
    ; K-`/id = `_
    ; K-wk  = λ s′ x → suc x
    -- ...
```

```
instance
  T : Kit T^M
  T = record
    { K-id/` = `_
    ; K-`/id = λ t → t
    ; K-wk  = λ s′ t → t ··· wk
    -- ...
```

In the K-wk field for the T Kit, we use the wk operation from the T Kit. The kit instance for that is automatically resolved using instance search. We can now turn our attention to the composition of maps.

4.2 Composition

Inside the ComposeKit record we can define a single, more general composition of a K_1-Map with a K_2-Map, resulting in a $(K_1 \sqcup K_2)$-Map:

```
record ComposeKit (K₁ : Kit M₁) (K₂ : Kit M₂) (K₁⊔K₂ : Kit M₃) : Set₁ where
  private instance _ = K₁; _ = K₂; _ = K₁⊔K₂
  field
    C-_&/···_ : S₁ ∋/⊢[ K₁ ] s → S₁ –[ K₂ ]→ S₂ → S₂ ∋/⊢[ K₁⊔K₂ ] s
```

The operation C-_&/···_ unifies lookup and traversal. It takes a variable or term x/t (according to K_1) and a renaming or substitution φ (according to K_2), and applies φ to x/t, yielding a variable or term (according to $K_1 \sqcup K_2$). From this operation, for which we again provide an opaque function version without the C-prefix, we derive map composition:

```
_;_ : S₁ –[ K₁ ]→ S₂ → S₂ –[ K₂ ]→ S₃ → S₁ –[ K₁⊔K₂ ]→ S₃
(φ₁ ; φ₂) _ x = (x & φ₁) &/··· φ₂
```

The map combination operator is implicitly parameterized over a Compose Kit. It takes an renaming or substitution φ_1 (according to K_1) and a renaming or substitution φ_2 (according to K_2) and composes it to a renaming or substitution (according to $K_1 \sqcup K_2$). When x & φ_1 yields a variable, we need to lookup the resulting variable in φ_2, or if we have a term, we traverse the term with φ_2. We also define versions of both the unified lookup–traversal operator and the composition operator where the Compose Kit is given explicitly: _;[_]_ φ_1 C φ_2 = ComposeKit._;_ C φ_1 φ_2 and _&/···[_]_ x/t C φ_2 = ComposeKit._&/···_ C x/t φ_2.

As with Kits, we now require a user-provided proof about traversal. We need the user to show that the chosen syntax and traversal function satisfy compositionality. This proof can only be established by induction on the syntax:

```
···-compositionality : ∀ {{K₁ : Kit M₁}} {{K₂ : Kit M₂}} {{K₃ : Kit M₃}} {{C : ComposeKit K₁ K₂ K₃}}
  (t : S₁ ⊢ s) (φ₁ : S₁ –[ K₁ ]→ S₂) (φ₂ : S₂ –[ K₂ ]→ S₃) →
  (t ··· φ₁) ··· φ₂ ≡ t ··· (φ₁ ; φ₂)
```

Finally, to cover the four possible instances of Compose Kits, we provide two generic instances that suffice (axiom proofs omitted):

```
instance
  C^R : {{K : Kit M}} → ComposeKit V K K
  C^R = record { C-_&/⋯_ = _&_ -- ...
```

```
instance
  C^S : {{K : Kit M}} {{C : ComposeKit K V K}} → ComposeKit T K T
  C^S = record { C-_&/⋯_ = _⋯_ -- ...
```

C^R is the Compose Kit for composition between a renaming and another K-Map. C^S is the Compose Kit for composition between a substitution and another K-Map, and assumes that composition of a K-Maps with renamings is already available.

The third parameter $K_1 \sqcup K_2$ of a Compose Kit can be understood as a functional dependency, uniquely determined by the choice of K_1 and K_2. It evaluates to V if and only if both K_1 and K_2 are V, otherwise it is T. In other words, only renamings compose to a renaming, while every other case yields a substitution. We can put this more algebraically: if φ_1 is a K_1-Map and φ_2 is a K_2-Map, then the result is a $(K_1 \sqcup K_2)$-Map, where $\sqcup$ refers to the join-semilattice on $\{V, T\}$ generated by $V < T$. One might ask why we do not simply compute $K_1 \sqcup K_2$. This is in fact possible and even necessary to define the σ-calculus rules in a way that still adheres to the three restrictions on the shape of rewrite rules.

4.3 Structural Kit Order

We can easily define a least upper bound operation on Modes that mirrors the behaviour of composing two maps with these Modes:

```
_⊔_ : Mode → Mode → Mode
V^M ⊔ V^M = V^M
V^M ⊔ T^M = T^M
T^M ⊔ V^M = T^M
T^M ⊔ T^M = T^M
```

By case analysis, all the join-semilattice laws are verified easily:

```
⊔-idem      : M ⊔ M             ≡ M
⊔-assoc     : (M₁ ⊔ M₂) ⊔ M₃ ≡ M₁ ⊔ (M₂ ⊔ M₃)
⊔-comm      : (M₁ ⊔ M₂)         ≡ (M₂ ⊔ M₁)
⊔-bot-right : M ⊔ V^M ≡ M
⊔-bot-left  : V^M ⊔ M ≡ M
⊔-top-right : M ⊔ T^M ≡ T^M
⊔-top-left  : T^M ⊔ M ≡ T^M
```

All laws can be embedded as reduction rules in Agda using REWRITE, except for ⊔-comm, which is evidently non-terminating. We then lift this operation from Modes to Kits themselves, using the least upper bound on Modes in the type of the Kit:

```
_⊔_ : (K₁ : Kit M₁) (K₂ : Kit M₂) → Kit (M₁ ⊔ M₂)
_⊔_ {V^M} {V^M} K₁ K₂ = V
_⊔_ {V^M} {T^M} K₁ K₂ = T
_⊔_ {T^M} {V^M} K₁ K₂ = T
_⊔_ {T^M} {T^M} K₁ K₂ = T
```

The same laws that hold for Modes, also hold for Kits:

```
⊔-idem      : {{K : Kit M}} → K ⊔ K ≡ K
⊔-assoc     : {{K₁ : Kit M₁}} {{K₂ : Kit M₂}} {{K₃ : Kit M₃}} →
  (K₁ ⊔ K₂) ⊔ K₃ ≡ K₁ ⊔ (K₂ ⊔ K₃)
⊔-comm      : {{K₁ : Kit M₁}} {{K₂ : Kit M₂}} →
  (K₁ ⊔ K₂) ≡ subst Kit (⊔-comm {M₂} {M₁}) (K₂ ⊔ K₁)
⊔-bot-right : {{K : Kit M}} → K ⊔ V ≡ K
⊔-bot-left  : {{K : Kit M}} → V ⊔ K ≡ K
⊔-top-right : {{K : Kit M}} → K ⊔ T ≡ T
⊔-top-left  : {{K : Kit M}} → T ⊔ K ≡ T
```

Note that the ⊔-comm law requires the transfer lemma in its type, since we could not rewrite ⊔-comm for Modes. All other laws are again added as rewrite rules. The Kit least upper bound laws follow by case distinction on their Modes. When one inspects a proof in detail, we get into trouble. As an example, consider ⊔-idem. The remaining laws follow in the same way:

```
⊔-idem {V^M} {{K}} = unique-V K
⊔-idem {T^M} {{K}} = unique-T K
```

If we split on the Mode M of the Kit K, we must prove in both cases that K is either V or T, according to the definition of _⊔_ and the chosen Mode. Here we run into a problem: K is inferred by instance resolution, and Agda cannot deduce that there is only one possible choice, V for V^M and T for T^M. Nor can the uniqueness of instances be assumed in general, since external code might declare additional instances. To resolve this, we restrict the Kit instances by requiring a field K-lock: Lock of type private data Lock : Set where unlock : Lock in the construction of a Kit. Because Lock is private, it can only be constructed from within the library. We can now postulate the uniqueness of Kits after manually verifying that only two instances exist and that we rely solely on Kits provided by instance resolution, never creating them locally:

```
private postulate
  unique-V : (K : Kit V^M) → V ≡ K
  unique-T : (K : Kit T^M) → T ≡ K
```

There also exists a version unique-K that computes whether a given K : Kit M is V or T, based on its Mode.

4.4 Computing Compose Kits

Our original goal was to compute the third parameter of a Compose Kit. To motivate this and, more generally, the computation of Compose Kits from Kits, we consider a first attempt of the defintion for the associativity law of the σ-calculus, which in our Kit-based setting corresponds to the associativity of map composition:

```
associativity :
  {{K₁ : Kit M₁}} {{K₂ : Kit M₂}} {{K₃ : Kit M₃}} {{K₄ : Kit M₄}} {{K₅ : Kit M₅}} {{K₆ : Kit M₆}}
  {{C₁ : ComposeKit K₁ K₂ K₃}} {{C₂ : ComposeKit K₃ K₄ K₅}} {{C₃ : ComposeKit K₁ K₆ K₅}} {{C₄ : ComposeKit K₂ K₄ K₆}}
```

```
(φ₁ : S₁ −[ K₁ ]→ S₂) (φ₂ : S₂ −[ K₂ ]→ S₃) (φ₃ : S₃ −[ K₄ ]→ S₄) →
(φ₁ ;[ C₁ ] φ₂) ;[ C₂ ] φ₃ ≡ φ₁ ;[ C₃ ] (φ₂ ;[ C₄ ] φ₃)
```

Here C_1 to C_6 denote the four Compose Kits that are required for the compositions. For clarity we apply them to the composition operators explicitly, although instance resolution would perform this step automatically. This yields the most general formulation of the associativity law for Kits, where arbitrary Kits are admitted as long as both sides reduce to a K_5-Map. This law, however, is not a valid rewrite rule. Both C_3 and C_4, required for the right side, appear only there, violating the condition that all arguments of a rewrite rule must occur on the left. Moreover, the law requires the instance argument K_6, used as the result of composing the K_2-Map with that of K_4, which is only required on the right. Since K_6 can be computed via the least upper bound operation defined earlier, the main difficulty is to compute C_3 and C_4.

One may attempt to define a function `_;ᴷ_ : (K₁ : Kit M₁) (K₂ : Kit M₂) → ComposeKit K₁ K₂ (K₁ ⊔ K₂)` for this purpose. This function can be readily defined, but its correctness relies not only on the uniqueness of Kit instances, but also on the uniqueness of Compose Kit instances:

```
private postulate
  unique-Cᴿ : {{K : Kit M}} (C : ComposeKit V K K) → C ≡ Cᴿ
  unique-Cˢ : {{K : Kit M}} {{C′ : ComposeKit K V K}} → (C : ComposeKit T K T) → C ≡ Cˢ
```

As before we reqiure a value of `Lock` in the creation of Compose Kits. By manual inspection we confirm that `Cᴿ` and `Cˢ` are the only instances and that none are created locally.

Yet the function `_;ᴷ_` is insufficiently general. Computing C_4 in the associativity law yields `C₄ = K₂ ;ᴷ K₄ = ComposeKit K₂ K₄ (K₂ ⊔ K₄)`. Consequently, C_3 evaluates to `C₃ = K₁ ;ᴷ (K₂ ⊔ K₄) = ComposeKit K₁ (K₂ ⊔ K₄) (K₁ ⊔ (K₂ ⊔ K₄))`. The final Map on right side is then a `K₁ ⊔ (K₂ ⊔ K₄)`-Map, whereas the left side is a K_5-Map. Adjusting the K_5 parameter in C_1 or C_2 to the result Kit of C_4 is not possible, since Compose Kits whose third parameter involves least upper bounds cannot be inferred by instance resolution[3]. We require an even more general, though unsafe, function:

```
{-# NON_COVERING #-}
_,_,_ : (K₁ : Kit M₁) (K₂ : Kit M₂) (K₃ : Kit M₃) → ComposeKit K₁ K₂ K₃
_,_,_ {Vᴹ} {Vᴹ} {Vᴹ} K₁ K₂ K₃ rewrite unique-K K₁ | unique-K K₂ | unique-K K₃ = Cᴿ
_,_,_ {Vᴹ} {Tᴹ} {Tᴹ} K₁ K₂ K₃ rewrite unique-K K₁ | unique-K K₂ | unique-K K₃ = Cᴿ
_,_,_ {Tᴹ} {Vᴹ} {Tᴹ} K₁ K₂ K₃ rewrite unique-K K₁ | unique-K K₂ | unique-K K₃ = Cˢ
_,_,_ {Tᴹ} {Tᴹ} {Tᴹ} K₁ K₂ K₃ rewrite unique-K K₁ | unique-K K₂ | unique-K K₃ = Cˢ
```

Since it is declared non-covering, evaluation may get stuck, yielding a value for a Compose Kit that should not exist. Thus, every use of this function requires manual verification. Computed Compose Kits are reduced to the actual instances, when applicable, using the rewrite rules `{{K : Kit M}} → V , K , K ≡ Cᴿ {{K = K}}` and `{{K : Kit M}} → T , K , T ≡ Cˢ {{K = K}} {{C = K , V , K}}`. This allows us to state a version of the associativity law eligible for rewriting:

```
associativity :
  {{K₁ : Kit M₁}} {{K₂ : Kit M₂}} {{K₃ : Kit M₃}} {{K₄ : Kit M₄}} {{K₅ : Kit M₅}}
  {{C₁ : ComposeKit K₁ K₂ K₃}} {{C₂ : ComposeKit K₃ K₄ K₅}}
```

[3]Adding instances that would allow the inference of Compose Kits with least upper bounds did not work out for us. In addition, Agda generally refused to infer such instances during higher-order pattern unification when checking if a rewrite rule applies.

```
(φ₁ : S₁ −[ K₁ ]→ S₂) (φ₂ : S₂ −[ K₂ ]→ S₃) (φ₃ : S₃ −[ K₄ ]→ S₄) →
-- SAFETY: Assume the incoming Compose Kits C₁ and C₂ are valid.
--         By case analysis on K₁, K₂, K₃, K₄ and K₅, the definition of _⊔_ and
--         the uniqueness of (Compose-) Kits, we know the outgoing Compose Kits
--         K₁ , (K₂ ⊔ K₄) , K₅ and K₂ , K₄ , (K₂ ⊔ K₄) are valid too. □
(φ₁ ; φ₂) ; φ₃ ≡ φ₁ ;[ K₁ , (K₂ ⊔ K₄) , K₅ ] (φ₂ ;[ K₂ , K₄ , (K₂ ⊔ K₄) ] φ₃)
```

We can now connect $K_1 \sqcup (K_2 \sqcup K_4)$ with K_5 in the outer Compose Kit on the right side. It remains to check that whenever the incoming Compose Kits C_1 and C_2 are valid, the outgoing ones, computed using our unsafe function, are as well. As an illustrative case, if $K_1 = .. = K_5 =$ V, then all Maps are renamings and the outgoing Compose Kits correspond to compositions of renamings. By exhaustively checking all 8 valid combinations of values for C_1 and C_2, we confirm that in each case the outgoing Compose Kits are valid. Inside the associativity proof, the remaining cases must be refuted under the assumption of unique incoming Compose Kits and the non-existance of invalid Compose Kits:

```
private postulate
  impossible-C^SKR : {{K : Kit M}} → ¬ ComposeKit T K V
  impossible-C^KSR : {{K : Kit M}} → ¬ ComposeKit K T V
  impossible-C^RRS : ¬ ComposeKit V V T
```

As before, each use of this postulate requires manual verification to avoid inconsistencies.

All other σ-calculus laws, generalized to Maps, follow the same structure as the associativity law. On the left side of the equations we rely on Kits and Compose Kits inferred via instance resolution, while on the right side we compute the resulting Kits and Compose Kits using the least upper bound operation together with the unsafe _,_,_ operator, where we manually verify that its use is sound. The proofs are carried out by case analysis on the Modes of the Kit instance arguments and rely on the uniqueness of (Compose-) Kits as well as the refutation of invalid ComposeKit Kits. Apart from these aspects, their structure closely resembles the familiar proofs for separate renamings and substitutions.

4.5 The Scoped and Multi-Sorted σ-Calculus with Kits and Maps

We are now ready to state the σ-calculus laws in the Kit-setting. Figure 5 lists the symbols and laws as they are used in Agdasubst. These are not the formulations from the library itself. For readability, we omit the Kit and ComposeKit instance arguments. Kits and ComposeKits carry data in the library, and Kits must be disambiguated on the type level by Modes. Here we simply assume that Kits act as markers, which allows us to elide the mode lattice laws. Similarly, Compose Kits serve as markers for admissible compositions and are given explicitly for every operator in square brackets.

The symbols are grouped into several categories. Sorts and Scopes provide intrinsic scoping at the type level. Variables are supplied by the library, while terms are user-defined, subject only to the requirement of a variable constructor in the appropriate form. We define the type for Kits and the variable and term Kit, together with the least upper bound operation. We then define the type of Maps and the four map primitives, parameterized by Kits. We specialize wk directly to a renaming in order to avoid duplication, since it can always be embedded as a substitution by composition with the identity substitution. When the Kit in the type of Maps is known to be V or T, we use the variables ρ and σ, respectively. From Kits we further obtain the combined type of variables-or-terms, together with the unified lookup/traversal operator. While there are separate type definitions for variables, terms, and variables-or-terms, there is only one type definition

Symbols

Sorts s
$\mathsf{Sort} : \mathsf{Set}$

.. object language sorts ..

Scopes S
$\mathsf{Scope} : \mathsf{Set}$

$[\,] \quad : \mathsf{Scope}$
$_::_ : \mathsf{Sort} \to \mathsf{Scope} \to \mathsf{Scope}$

Variables x
$_\ni_ : \mathsf{Scope} \to \mathsf{Sort} \to \mathsf{Set}$

$\mathsf{zero} : (s :: S) \ni s$
$\mathsf{suc} \quad : S \ni s \to (s' :: S) \ni s$

Terms t
$_\vdash_ : \mathsf{Scope} \to \mathsf{Sort} \to \mathsf{Set}$

$`_ : S \ni s \to S \vdash s$

.. object language ..

Kits K
$\mathsf{Kit} : \mathsf{Set}$

$\mathsf{V} \quad : \mathsf{Kit}$
$\mathsf{T} \quad : \mathsf{Kit}$
$_\sqcup_ : \mathsf{Kit} \to \mathsf{Kit} \to \mathsf{Kit}$

Maps φ
$_-[_]\to_ : \mathsf{Scope} \to \mathsf{Kit} \to \mathsf{Scope} \to \mathsf{Set}$

$\mathsf{id} \quad : S -[\, K \,]\to S$
$\mathsf{wk} : S -[\, \mathsf{V} \,]\to (s :: S)$
$_\cdot_ : S_2 \ni/\vdash[\, K \,]\ s \to S_1 -[\, K \,]\to S_2 \to (s :: S_1) -[\, K \,]\to S_2$
$_;_ : S_1 -[\, K_1 \,]\to S_2 \to S_2 -[\, K_2 \,]\to S_3 \to S_1 -[\, K_3 \,]\to S_3$

Variables or Terms x/t
$_\ni/\vdash[_]_ : \mathsf{Scope} \to \mathsf{Kit} \to \mathsf{Sort} \to \mathsf{Set}$

$_\&/\cdots_ : S_1 \ni/\vdash[\, K_1 \,]\ s \to S_1 -[\, K_2 \,]\to S_2 \to S_2 \ni/\vdash[\, K_3 \,]\ s$

Reduction

Kit Laws

$\mathsf{bot}^R \ : K \sqcup \mathsf{V} \equiv K$
$\mathsf{bot}_l \ : \mathsf{V} \sqcup K \equiv K$
$\mathsf{top}^R \ : K \sqcup \mathsf{T} \equiv \mathsf{T}$
$\mathsf{top}_l \ : \mathsf{T} \sqcup K \equiv \mathsf{T}$
$\mathsf{idem} : K \sqcup K \equiv K$
$\mathsf{assoc} : (K_1 \sqcup K_2) \sqcup K_3 \equiv K_1 \sqcup (K_2 \sqcup K_3)$

Definitional Laws

$\mathsf{img}^V : S \ni/\vdash[\, \mathsf{V} \,]\ s \equiv S \ni s$
$\mathsf{img}^T : S \ni/\vdash[\, \mathsf{T} \,]\ s \equiv S \vdash s$

$\mathsf{id}^S : x \,\&/\cdots[\, \mathsf{V} , \mathsf{T} , \mathsf{T} \,]\ \mathsf{id}[\, \mathsf{T} \,] \equiv `\, x$
$\mathsf{wk} : x \,\&/\cdots[\, \mathsf{V} , \mathsf{V} , \mathsf{V} \,]\ \mathsf{wk} \equiv \mathsf{suc}\ x$
$\mathsf{ext}_0 \ : \mathsf{zero} \,\&/\cdots[\, \mathsf{V} , K , K \,]\ (x/t \cdot [\, K \,]\ \varphi) \equiv x/t$
$\mathsf{ext}_s \ : \mathsf{suc}\ x' \,\&/\cdots[\, \mathsf{V} , K , K \,]\ (x/t \cdot [\, K \,]\ \varphi) \equiv$
$\quad x' \,\&/\cdots[\, \mathsf{V} , K , K \,]\ \varphi$
$\mathsf{comp} : x \,\&/\cdots[\, \mathsf{V} , \mathsf{V} , \mathsf{V} \,]\ (\rho_1 ;[\, \mathsf{V} , \mathsf{V} , \mathsf{V} \,]\ \rho_2) \equiv$
$\quad (x \,\&/\cdots[\, \mathsf{V} , \mathsf{V} , \mathsf{V} \,]\ \rho_1) \,\&/\cdots[\, \mathsf{V} , \mathsf{V} , \mathsf{V} \,]\ \rho_2$

Definitional Composition Specialization Laws

$\mathsf{comp}_r\text{–}\mathsf{id}^S \ : x \,\&/\cdots[\, \mathsf{V} , \mathsf{T} , \mathsf{T} \,]\ (\rho_1 ;[\, \mathsf{V} , \mathsf{T} , \mathsf{T} \,]\ \mathsf{id}[\, \mathsf{T} \,]) \equiv$
$\quad `\,(x \,\&/\cdots[\, \mathsf{V} , \mathsf{V} , \mathsf{V} \,]\ \rho_1)$
$\mathsf{comp}_l\text{–}\mathsf{id}^S \ : x \,\&/\cdots[\, \mathsf{V} , \mathsf{T} , \mathsf{T} \,]\ (\mathsf{id}[\, \mathsf{T} \,] ;[\, \mathsf{T} , \mathsf{V} , \mathsf{T} \,]\ \rho_2) \equiv$
$\quad `\,(x \,\&/\cdots[\, \mathsf{V} , \mathsf{V} , \mathsf{V} \,]\ \rho_2)$
$\mathsf{comp}_l\text{–}\mathsf{wk} \ : x \,\&/\cdots[\, \mathsf{V} , K , K \,]\ (\mathsf{wk} ;[\, \mathsf{V} , K , K \,]\ \varphi') \equiv$
$\quad \mathsf{suc}\ x \,\&/\cdots[\, \mathsf{V} , K , K \,]\ \varphi'$
$\mathsf{comp}_l\text{–}\mathsf{ext}_0 : \mathsf{zero} \,\&/\cdots[\, \mathsf{V} , K_3 , K_3 \,]\ ((x/t_1 \cdot [\, K_1 \,]\ \varphi_1) ;[\, K_1 , K_2 , K_3 \,]\ \varphi_2) \equiv$
$\quad x/t \,\&/\cdots[\, K_1 , K_2 , K_3 \,]\ \varphi_2$
$\mathsf{comp}_l\text{–}\mathsf{ext}_s : \mathsf{suc}\ x' \,\&/\cdots[\, \mathsf{V} , K_3 , K_3 \,]\ ((x/t_1 \cdot [\, K_1 \,]\ \varphi_1) ;[\, K_1 , K_2 , K_3 \,]\ \varphi_2) \equiv$
$\quad x' \,\&/\cdots[\, \mathsf{V} , K_3 , K_3 \,]\ (\varphi_1 ;[\, K_1 , K_2 , K_3 \,]\ \varphi_2)$

Interaction Laws

$\mathsf{comp\text{-}id}_l \ : \mathsf{id}[\, K \,] ;[\, K , K , K \,]\ \varphi \equiv \varphi$
$\mathsf{comp\text{-}id}_r \ : \varphi ;[\, K , K , K \,]\ \mathsf{id}[\, K \,] \equiv \varphi$
$\mathsf{associativity} : (\varphi_1 ;[\, K_1 , K_2 , K_3 \,]\ \varphi_2) ;[\, K_3 , K_4 , K_5 \,]\ \varphi_4 \equiv$
$\quad \varphi_1 ;[\, K_1 , (K_2 \sqcup K_4) , K_5 \,]\ (\varphi_2 ;[\, K_2 , K_4 , (K_2 \sqcup K_4) \,]\ \varphi_4)$
$\mathsf{distributivity} : (x/t_1 \cdot [\, K_1 \,]\ \varphi_1) ;[\, K_1 , K_2 , K_3 \,]\ \varphi_2 \equiv$
$\quad (x/t_1 \,\&/\cdots[\, K_1 , K_2 , K_3 \,]\ \varphi_2) \cdot [\, K_3 \,]\ (\varphi_1 ;[\, K_1 , K_2 , K_3 \,]\ \varphi_2)$
$\mathsf{interact} \ : \mathsf{wk} ;[\, \mathsf{V} , K , K \,]\ (x/t \cdot [\, K \,]\ \varphi) \equiv \varphi$
$\eta\text{–}\mathsf{id} \ : \mathsf{zero} \cdot [\, \mathsf{V} \,]\ \mathsf{wk} \equiv \mathsf{id}[\, \mathsf{V} \,]$
$\eta\text{–}\mathsf{law} \ : (\mathsf{zero} \,\&/\cdots[\, \mathsf{V} , K , K \,]\ \varphi') \cdot [\, K \,]\ (\mathsf{wk} ;[\, \mathsf{V} , K , K \,]\ \varphi') \equiv \varphi'$

Traversal Laws

$\mathsf{var} : (`\, x) \,\&/\cdots[\, \mathsf{T} , K , \mathsf{T} \,]\ \varphi \equiv$
$\quad (x \,\&/\cdots[\, \mathsf{V} , K , K \,]\ \varphi) \,\&/\cdots[\, K , \mathsf{T} , \mathsf{T} \,]\ \mathsf{id}[\, \mathsf{T} \,]$

.. object language traversal ..

Monad Laws

$\mathsf{right\text{-}id} \ : x/t_1 \,\&/\cdots[\, K_1 , K_2 , K_1 \,]\ \mathsf{id}[\, K_2 \,] \equiv x/t_1$
$\mathsf{compositionality} : (x/t_1 \,\&/\cdots[\, K_1 , K_2 , K_3 \,]\ \varphi_1) \,\&/\cdots[\, K_3 , K_4 , \mathsf{T} \,]\ \varphi_4 \equiv$
$\quad x/t_1 \,\&/\cdots[\, K_1 , K_2 \sqcup K_4 , \mathsf{T} \,]\ (\varphi_1 ;[\, K_2 , K_4 , (K_2 \sqcup K_4) \,]\ \varphi_2)$

Coincidence Laws

$\mathsf{coincidence} : t \,\&/\cdots[\, \mathsf{T} , \mathsf{T} , \mathsf{T} \,]\ (\varphi ;[\, K , \mathsf{T} , \mathsf{T} \,]\ \mathsf{id}[\, \mathsf{T} \,]) \equiv$
$\quad t \,\&/\cdots[\, \mathsf{T} , K , \mathsf{T} \,]\ \varphi$
$\mathsf{coincidence\text{–}fold}^K : t \,\&/\cdots[\, \mathsf{T} , \mathsf{T} , \mathsf{T} \,]\ ((x/t \,\&/\cdots[\, K , \mathsf{T} , \mathsf{T} \,]\ \mathsf{id}[\, \mathsf{T} \,]) \cdot (\varphi ;[\, K , \mathsf{T} , \mathsf{T} \,]\ \mathsf{id}[\, \mathsf{T} \,])) \equiv$
$\quad (t \,\&/\cdots[\, \mathsf{T} , K , \mathsf{T} \,]\ (x/t \cdot \varphi))$
$\mathsf{coincidence\text{–}fold}^T : t \,\&/\cdots[\, \mathsf{T} , \mathsf{T} , \mathsf{T} \,]\ ((t' \,\&/\cdots[\, \mathsf{T} , K , \mathsf{T} \,]\ \varphi) \cdot (\varphi ;[\, K , \mathsf{T} , \mathsf{T} \,]\ \mathsf{id}[\, \mathsf{T} \,])) \equiv$
$\quad (t \,\&/\cdots[\, \mathsf{T} , \mathsf{T} , \mathsf{T} \,]\ (t' \cdot (\mathsf{id}[\, \mathsf{T} \,]))) \,\&/\cdots[\, \mathsf{T} , K , \mathsf{T} \,]\ \varphi$

Fig. 5. The Scoped and Multi-Sorted σ-Calculus with Kits and Maps.

for Maps, specialized via the Kit index. There is no technical reason not to unify variables and terms in the same way. However, unlike renamings, substitutions, and maps, terms are user-supplied, whereas variables are library-defined and thus qualify for a distinct type definition.

We already discussed the **Kit Laws** in Section 4.3, so we now turn to the remaining ones. We begin with the type-level reduction laws img^V and img^T inside the **Definitional Laws**, which rewrite the type of variables-or-terms to either the type of variables or the type of terms when the Kit index is known to be V or T, respectively.

Next comes the definition of lookup on the identity substitution id^S. It is given in terms of the unified lookup/traversal operator. In fact, we never use the specialized lookup or traversal operators at all. We considered keeping the specialized operators, but this would require duplicating many laws, once for the specialized operators and once for the general one[4]. Identity renaming does not require its own law, since it is already covered by the now generalized right-id from the **Monad Laws**, which handles both a term traversed by an identity substitution and a variable looked up in the identity renaming.

The laws wk, ext_0, and ext_s are the direct generalizations of their familiar counterparts for renaming and substitution. The law comp, however, is more subtle. As shown in Figure 5, it only reduces when both φ_1 and φ_2 are renamings. This restriction is necessary because of a fundamental duality between the definition of composition (via the unified lookup/traversal operator) and the compositionality law in full generality:

comp−general :

$$x/t \;\&/\cdots[\; K_3 \,,\, K_4 \,,\, K_5 \;]\; (\varphi_1 \;;[\; K_1 \,,\, K_2 \,,\, K_3 \;]\; \varphi_2) \equiv (x/t \;\&/\cdots[\; K_1 \,,\, K_2 \,,\, K_3 \;]\; \varphi_1) \;\&/\cdots[\; K_3 \,,\, K_4 \,,\, K_5 \;]\; \varphi_2$$

compositionality−general :

$$(x/t_1 \;\&/\cdots[\; K_1 \,,\, K_2 \,,\, K_3 \;]\; \varphi_1) \;\&/\cdots[\; K_3 \,,\, K_4 \,,\, K_5 \;]\; \varphi_4 \equiv x/t_1 \;\&/\cdots[\; K_1 \,,\, K_2 \sqcup K_4 \,,\, K_5 \;]\; (\varphi_1 \;;[\; K_2 \,,\, K_4 \,,\, (K_2 \sqcup K_4) \;]\; \varphi_2)$$

Viewed side by side, these laws are exact duals. If both were admitted into the rewrite system, they would trigger infinite loops. This is no accident, intuitively, one law pushes variables into maps, while the other pushes maps into other maps, away from terms.

To avoid nontermination, we adopt the following heuristic:

- In variable position ($K_5 = \mathsf{V}$) use the definition of composition. In this case all other indices $K_1, \ldots, K_4$ are forced to V, and x/t must be a variable. This yields exactly the specialized comp law shown in the figure.
- In term position ($K_5 = \mathsf{T}$) use compositionality. Here no restrictions on the other indices arise, so the law is preserved in most of its generality and appears under the **Monad Laws**.

If neither condition applies (i.e. we do not know whether we are dealing with a variable or a term and K is a variable), the user must explicitly choose from the fully general laws which law to apply, either by equational reasoning or by using rewrite. With this heuristic the comp law is rather restricted. We now miss cases where we would make progress by applying the comp law, when φ_1 and φ_2 are not both renamings. Hence, we consider all situations where progress would be made directly after applying comp, by specializing on φ_1 or φ_2. These appear under the name **Definitional Composition Specialization Laws**.

The **Interaction Laws** are the direct generalizations of their original σ-calculus counterparts to Kits. Next come the **Traversal Laws**. Since we are independent of the object syntax except for the required variable constructor, we only show the law specific to the traversal of a variable. When a variable constructor is applied to a map, the variable it holds is mapped accordingly. To turn the result back into a term, as it may otherwise be either a variable or a term, depending on the map, we subsequently apply the identity substitution.

[4]There are certainly situations where such duplication could help, but in our case studies, with the addition of the definitional composition specialization laws, we had no need for laws specialized to lookup or traversal.

To illustrate another case for traversal, consider an object language that supports λ-abstractions. The corresponding user-supplied traversal function, needed to construct the term Kit, would then include a clause $(\lambda\mathsf{x}\ e)\cdots\ \varphi = \lambda\mathsf{x}\ (e \cdots (\varphi \uparrow _))$. However, this clause does not directly corresponds to a traversal law. It must first be lifted from the traversal operator to the unified lookup/traversal operator: $(\lambda\mathsf{x}\ e)\ \&/\cdots\ \varphi \equiv \lambda\mathsf{x}\ (e\ \&/\cdots\ (\varphi \uparrow _))$. This equality can be proven by reflexivity, when we unfold the definition of the unfied lookup/traversal operator. It can then be added as a rewrite rule.

Finally, the **Coincidence Laws** generalize from renamings to maps and allow maps to be pulled out of a substitution when feasible. These laws are not a complete account of all possible ways to pull maps out of substitutions. The search for more sophisticated laws that cover additional cases where this can be done would certainly be worthwhile.

4.6 Discussion

The generalization of the σ-calculus laws required unsafe computation and a careful formulation of the laws so that they remain valid rewrite rules while still allowing the inference of Kits and ComposeKits to be offloaded to the instance resolution algorithm. The key difficulty in adapting the laws to the Kit-setting stems from the duality between the general versions of the comp and the compositionality laws. Our solution is a heuristic that resolves the conflict by restricting the laws to cases where it is known whether we are in variable or term position. In particular, when we dont know, the direction in which the laws should flow becomes ambiguous. In such situations, the user must resort to manual application of the laws. In contrast to the laws from Section 3.2, the Kit-based laws allow us to establish properties directly for Maps, without the need to first carry out separate but structurally equal proofs for renamings and substitutions.

5 Case Studies

We will discuss two examples demonstrating the usage of the Agdasubst library. The first concerns proofs of subject reduction for System F with Subtyping. We begin by presenting the proof using the laws from Section 3.2, and then revisit it using the Kit-based approach from Section 4.5. Afterwards, we extend the Agdasubst version with subtyping. The second example, based on CITE, provides an instantiation for a generic universe of scoped syntax.

5.1 System F (+ Subtyping)

Recall the multi-sorted and scoped syntax from Figure 2. We briefly outline a proof of subject reduction, first using the laws from Section 3.2, and then using the Agdasubst library. The type system and semantics for both cases are essentially the same, and we present them once in Figure 6. In both settings, we make use of functions $\mathsf{weaken} : S \vdash s \to (s' :: S) \vdash s$ and $_[_] : (s' :: S) \vdash s \to S \vdash s' \to S \vdash s$
$t\ [\ t'\ ] = t \cdots^{S} (t' \cdot^{S} \mathsf{id}^{S})$. They are straightforward to define from the primitves in both cases. Within the typing relation, we rely on the lookup relation $_\ni_:_ : \mathsf{Ctx}\ S \to S \ni s \to S :\vdash s \to \mathsf{Set}$ for retrieving the type of a variable from the typing context. The type of the lookup and typing relation uses the operator $_:\vdash_$, defined as:

$\uparrow^{t}_ : \mathsf{Sort} \to \mathsf{Sort}$
$\uparrow^{t}\ \mathsf{expr} = \mathsf{type}$
$\uparrow^{t}\ \mathsf{type} = \mathsf{kind}$
$\uparrow^{t}\ \mathsf{kind} = \mathsf{kind}$

$_:\vdash_ : \mathsf{Scope} \to \mathsf{Sort} \to \mathsf{Set}$
$S :\vdash s = S \vdash (\uparrow^{t}\ s)$

```
data _⊢_:_ : Ctx S → S ⊢ s → S :⊢ s → Set where
  ⊢` : ∀ {x : S ∋ s} {t} →
    Γ ∋ x : t →
    --------------
    Γ ⊢ (` x) : t
  ⊢λ :
    (t ::t Γ) ⊢ e : (weaken t′) →
    --------------------------
    Γ ⊢ (λx e) : (t ⇒ t′)
  ⊢Λ :
    (k ::t Γ) ⊢ e : t →
    --------------------------
    Γ ⊢ (Λα e) : (∀[α: k ] t)
  ⊢· :
    Γ ⊢ e1 : (t1 ⇒ t2) →
    Γ ⊢ e2 : t1 →
    --------------------
    Γ ⊢ (e1 · e2) : t2
  ⊢• :
    Γ ⊢ e : (∀[α: k ] t′) →
    Γ ⊢ t : k →
    (k ::t Γ) ⊢ t′ : k′ →
    -------------------------
    Γ ⊢ (e • t) : (t′ [ t ])
  ⊢★ : {t : S ⊢ type} →
    ---------
    Γ ⊢ t : ★
```

```
data Val : S ⊢ expr → Set where
  vλ : Val (λx e)
  vΛ : Val (Λα e)

data _↪_ : S ⊢ expr → S ⊢ expr → Set where
  β-λ :
    Val e2 →
    ----------------------------
    ((λx e1) · e2) ↪ (e1 [ e2 ])
  β-Λ :
    -------------------------
    ((Λα e) • t) ↪ (e [ t ])
  ξ-·1 :
    e1 ↪ e →
    --------------------
    (e1 · e2) ↪ (e · e2)
  ξ-·2 :
    e2 ↪ e →
    Val e1 →
    --------------------
    (e1 · e2) ↪ (e1 · e)
  ξ-• :
    e ↪ e′ →
    -------------------
    (e • t) ↪ (e′ • t)
```

Fig. 6. Typing and semantics for System F.

Separate Renamings and Substitutions

We have already seen the instantiation of the σ-calculus for System F in Section 3.2, where renamings and substitutions are treated separately. To establish subject reduction, we first prove that renaming preserves typing, and then, using this result in the proof for the typed lifiting of a substitution, show that general substitution preserves typing as well.

```
_⊢⋯ᴿ_ : ∀ {e : S₁ ⊢ s} {t : S₁ :⊢ s} →
  ρ : Γ₁ →ᴿ Γ₂ →
  Γ₁ ⊢ e : t →
  Γ₂ ⊢ (e ⋯ᴿ ρ) : (t ⋯ᴿ ρ)
⊢ρ ⊢⋯ᴿ (⊢` ⊢x) =
  ⊢` (⊢ρ _ _ _ ⊢x)
⊢ρ ⊢⋯ᴿ (⊢λ ⊢e)      =
  ⊢λ ((⊢↑ᴿ ⊢ρ _) ⊢⋯ᴿ ⊢e)
⊢ρ ⊢⋯ᴿ (⊢Λ ⊢e)      =
  ⊢Λ ((⊢↑ᴿ ⊢ρ _) ⊢⋯ᴿ ⊢e)
⊢ρ ⊢⋯ᴿ (⊢· ⊢e₁ ⊢e₂) =
  ⊢· (⊢ρ ⊢⋯ᴿ ⊢e₁) (⊢ρ ⊢⋯ᴿ ⊢e₂)
⊢ρ ⊢⋯ᴿ (⊢• ⊢e ⊢t ⊢t') =
  ⊢• (⊢ρ ⊢⋯ᴿ ⊢e) (⊢ρ ⊢⋯ᴿ ⊢t)((⊢↑ᴿ ⊢ρ _) ⊢⋯ᴿ ⊢t')
⊢ρ ⊢⋯ᴿ ⊢★           =
  ⊢★
```

```
_⊢⋯ˢ_ : ∀ {σ : S₁ →ˢ S₂} {e : S₁ ⊢ s} {t : S₁ :⊢ s} →
  σ : Γ₁ →ˢ Γ₂ →
  Γ₁ ⊢ e : t →
  Γ₂ ⊢ (e ⋯ˢ σ) : (t ⋯ˢ σ)
⊢σ ⊢⋯ˢ (⊢` ⊢x) =
  ⊢σ _ _ _ ⊢x
_⊢⋯ˢ_ {σ = σ} ⊢σ (⊢λ ⊢e)      =
  ⊢λ (_⊢⋯ˢ_ {σ = σ ↑ˢ _} (⊢↑ˢ {σ = σ} ⊢σ _) ⊢e)
_⊢⋯ˢ_ {σ = σ} ⊢σ (⊢Λ ⊢e)      =
  ⊢Λ (_⊢⋯ˢ_ {σ = σ ↑ˢ _} (⊢↑ˢ {σ = σ} ⊢σ _) ⊢e)
_⊢⋯ˢ_ {σ = σ} ⊢σ (⊢· ⊢e₁ ⊢e₂) =
  ⊢· (_⊢⋯ˢ_ {σ = σ} ⊢σ ⊢e₁) (_⊢⋯ˢ_ {σ = σ} ⊢σ ⊢e₂)
_⊢⋯ˢ_ {σ = σ} ⊢σ (⊢• ⊢e ⊢t ⊢t') =
  ⊢• (_⊢⋯ˢ_ {σ = σ} ⊢σ ⊢e) (_⊢⋯ˢ_ {σ = σ} ⊢σ ⊢t)
     (_⊢⋯ˢ_ {σ = σ ↑ˢ _} (⊢↑ˢ {σ = σ} ⊢σ _) ⊢t')
_⊢⋯ˢ_ ⊢σ ⊢★ =
  ⊢★
```

Both proofs rely on the types for well-typed renamings `_:_→ᴿ_ : S₁ →ᴿ S₂ → Ctx S₁ → Ctx S₂ → Set` and well-typed substitutions `_:_→ˢ_ : S₁ →ˢ S₂ → Ctx S₁ → Ctx S₂ → Set`, whose definitions we omit here. Within the respective cases, we look up the well-typed variable in the typed renaming/substitution, and when going under a binder we use the typed lifting of renamings and substitutions. Thanks to the automatic application of the σ-calculus laws, both proofs follow directly by induction, even in the cases where we go under a binder. Using the two results, we can then prove subject reduction for System F:

```
subject-reduction :
  Γ ⊢ e : t →
  e ↪ e' →
  Γ ⊢ e' : t
subject-reduction (⊢· {e₂ = e₂} (⊢λ ⊢e₁) ⊢e₂) (β-λ v₂) = _⊢⋯ˢ_ {σ = e₂ ·ˢ idˢ} (⊢[] ⊢e₂) ⊢e₁
subject-reduction (⊢• {t = t} (⊢Λ ⊢e) ⊢t ⊢t') β-Λ       = _⊢⋯ˢ_ {σ = t ·ˢ idˢ} (⊢[] ⊢t) ⊢e
subject-reduction (⊢· ⊢e₁ ⊢e₂)  (ξ-·₁ e₁↪e)            = ⊢· (subject-reduction ⊢e₁ e₁↪e) ⊢e₂
subject-reduction (⊢· ⊢e₁ ⊢e₂)  (ξ-·₂ e₂↪e x)          = ⊢· ⊢e₁ (subject-reduction ⊢e₂ e₂↪e)
subject-reduction (⊢• ⊢e ⊢t ⊢t') (ξ-• e↪e')             = ⊢• (subject-reduction ⊢e e↪e') ⊢t ⊢t'
```

All cases follow without requiring any explicit application of substitution lemmas. In the *beta*-reduction cases, we use the substitution preserves typing lemma.

Using Agdasubst

We first instantiate the library by providing the traversal function and the syntax dependent laws. The traversal function for maps in System F is given by:

```
private _⋯_ : ∀ {{K : Kit M}} → S₁ ⊢ s → S₁ −[ K ]→ S₂ → S₂ ⊢ s
(` x)        ⋯ φ = `/id (x & φ)
(λx e)       ⋯ φ = λx (e ⋯ (φ ↑ _))
(e₁ · e₂)    ⋯ φ = (e₁ ⋯ φ) · (e₂ ⋯ φ)
(t₁ ⇒ t₂)    ⋯ φ = (t₁ ⋯ φ) ⇒ (t₂ ⋯ φ)
(Λα e)       ⋯ φ = Λα (e ⋯ (φ ↑ _))
(∀[α: k ] t) ⋯ φ = ∀[α: (k ⋯ φ) ] (t ⋯ (φ ↑ _))
(e • t)      ⋯ φ = (e ⋯ φ) • (t ⋯ φ)
★            ⋯ φ = ★
```

In the case of variables, we need to turn the variable into either a variable or a term. We can now proceed to prove the monad laws:

```
{-# REWRITE id↑≡id #-}
⋯-right-id : ∀ {{K : Kit M}} (t : S ⊢ s) →
  t ⋯ id ≡ t
```

```
{-# REWRITE dist−↑−; #-}
⋯-compositionality :
  ∀ {{K₁ : Kit M₁}} {{K₂ : Kit M₂}} {{K₃ : Kit M₃}}
    {{C : ComposeKit K₁ K₂ K₃}} →
    (t : S₁ ⊢ s) (φ₁ : S₁ −[ K₁ ]→ S₂) (φ₂ : S₂ −[ K₂ ]→ S₃) →
    (t ⋯ φ₁) ⋯ φ₂ ≡ t ⋯ (φ₁ ; φ₂)
```

For right identity, in the case where we go under a binder, we need to show that (id/` (zero {s = s} {S = S})) · (id ;wk) ≡ id, and for compositionality we require ((φ₁ ↑ s) ; (φ₂ ↑ s)) ≡ ((φ₁ ; φ₂) ↑ s). Both of these laws id↑≡id and dist−↑−; are provided by the library and can already be rewritten. Both follow from the rules of the σ-calculus and thus do not cause any issues when rewritten. With these two laws applied, all cases in the proofs of both lemmas, including those where we go under a binder, follow directly by structural induction without any need for manual application of lemmas.

In the end we can re-define lookup and traversal in terms of the unified lookup/traversal operator:

```
_&_ : {{K : Kit M}} → S₁ ∋ s → S₁ −[ K ]→ S₂ → S₂ ∋/⊢[ K ] s
_&_ = _&/⋯_
```

```
_⋯_ : {{K : Kit M}} → S₁ ⊢ s → S₁ −[ K ]→ S₂ → S₂ ⊢ s
_⋯_ {{K}} = let instance _ = K ;ᴷ V in _&/⋯_
```

To prove that renaming and substitution preserve typing in a single lemma, we make use of Typing Kits, following the approach of Saffrich [18]. Typing Kits abstract over typed renamings and substitutions depending on a Kit, in a way analogous to how Maps abstract over renamings and substitutions based on a Kit. Within the library, they are instantiated exactly twice, once for the variables Kit (i.e. typed renamings) and once for term Kit (i.e. typed substitutions). For a more detailed description, we refer the reader to the already mentioned work. The unified proof that maps preserve typing mirrors the shape of the separate proofs:

```
_⊢⋯_ : ∀ {{K : Kit M}} {{TK : TypingKit K}}
  {Γ₁ : Ctx S₁} {Γ₂ : Ctx S₂}
```

```
  {e : S₁ ⊢ s} {t : S₁ :⊢ s} {φ : S₁ −[ K ]→ S₂} →
  Γ₁ ⊢ e : t →
  Γ₂ ∋*/⊢*[ TK ] φ : Γ₁ →
  Γ₂ ⊢ (e ⋯ φ) : (t ⋯ φ)
⊢‘ {x = x} ⊢x ⊢⋯ ⊢φ = ⊢‘/id (⊢φ x _ ⊢x)
⊢λ ⊢e      ⊢⋯ ⊢φ = ⊢λ (⊢e ⊢⋯ (⊢φ ∋↑/⊢↑ _))
⊢Λ ⊢e      ⊢⋯ ⊢φ = ⊢Λ (⊢e ⊢⋯ (⊢φ ∋↑/⊢↑ _))
⊢· ⊢e₁ ⊢e₂ ⊢⋯ ⊢φ = ⊢· (⊢e₁ ⊢⋯ ⊢φ) (⊢e₂ ⊢⋯ ⊢φ)
⊢• ⊢e ⊢t ⊢t′ ⊢⋯ ⊢φ = ⊢• (⊢e ⊢⋯ ⊢φ) (⊢t ⊢⋯ ⊢φ) (⊢t′ ⊢⋯ (⊢φ ∋↑/⊢↑ _))
⊢★         ⊢⋯ ⊢φ = ⊢★
```

Agdasubst automatically solves all equations occuring from lifting. In the variable case, we additionally need to convert a typed variable or term into a typed term. The proof of subject reduction is exactly the same as the one presented before. The only difference is that, instead of using the substitution preserves typing lemma, we use the more general lemma that maps preserve typing in the cases of β-reductions:

```
subject-reduction (⊢· {e₂ = e₂} (⊢λ ⊢e₁) ⊢e₂) (β-λ _) = _⊢⋯_ {φ = e₂ · id} ⊢e₁ (⊢[] ⊢e₂)
subject-reduction (⊢• {t = t} (⊢Λ ⊢e) ⊢t ⊢t′) β-Λ     = _⊢⋯_ {φ = t · id} ⊢e (⊢[] ⊢t)
```

Subtyping

When we extend System F with subtyping[6], substitution does not generally preserve typing because type variables must respect subtyping bounds. To address this, we can encode a type variable binding $\alpha < t$ as two bindings, $\alpha : \star$ and a constraint $c : (\alpha < t)$. This restricted, first-class-constraint–style encoding is isomorphic to the original formalization and makes substitution-preserves-typing hold again. With this formulation, violating a bound simply yields a context with an unsatisfiable constraint [18].

The extension with subtyping substantially complicates the proof of subject reduction. In this setting we must reason about an injective subset of renamings as well as about various inversion lemmas that arise due to the additional subtyping constraints. We also require the fact that renaming preserves reduction. All additional lemmas together with the subject reduction theorem itself can be established within Agdasubst without any need for manual application of substitution lemmas and can be found in the supplement.

5.2 Generic Syntax

The class of object languages supported by Agdasubst can be described the definition of a generic syntax à la Allais et al. [3], adapted to the intrinsically scoped setting [18]:

```
data Desc : Set₁ where
  ‘σ : (A : Set) → (A → Desc) → Desc
  ‘X : Scope → Sort → Desc → Desc
  ‘■ : Sort → Desc

⟦_⟧ : Desc → Scoped → Scoped
⟦ ‘σ A d    ⟧ X S s = Σ[ a ∈ A ] (⟦ d a ⟧ X S s)
⟦ ‘X S′ s′ d ⟧ X S s = X (S′ ++ S) s′ × ⟦ d ⟧ X S s
⟦ ‘■ s′ ⟧ X S s = s ≡ s′

data Tm (d : Desc) : Scoped where
  ‘var : S ∋ s → Tm d S s
  ‘con : ⟦ d ⟧ (Tm d) S s → Tm d S s
```

This definition includes constructors for sums ‘σ and for object language constructors that introduce 0 to n binders ‘X.[5] All required laws for instantiating Agdasubst can be proven once and for all. Users of the library can define their own syntax directly from the building blocks of the generic syntax, without implementing traversals or reproving syntax-specific laws. For example, the sigma-calculus laws apply automatically to the following encoding of the lambda calculus:

```
desc : Desc
desc = ‘σ Label λ where
  [λ] → ‘X (expr :: []) expr (‘■ expr)
  [·] → ‘X [] expr (‘X [] expr (‘■ expr))

pattern ‘_ x   = ‘var x
pattern λx_ e = ‘con ([λ] , e , (refl , refl))
pattern _·_ e₁ e₂ = ‘con ([·] , e₁ , e₂ , (refl , refl))
```

Here, the generic encoding has a single sort of expressions, data Sort : Set where expr : Sort, and two labels, one for abstraction and one for application, data Label : Set where [λ] [·] : Label. While the generic approach is very convenient, the main drawback is that Agda sometimes unfolds the pattern definitions inside holes, leaking details of the encoding, which results in proof states that are hard to read.

6 Related Work

Most of this work is inspired by and builds upon the work theoretical work conducted in the development of Autosubst [20]. Autosubst, like Agdasubst, uses the laws of the σ-calculus to automatically solve equations between terms with substitutions in the Rocq proof assistant. With version 2 [23], Autosubst also introduced scoped syntax and first-class renamings, making their law set comparable to the one presented in Section 3.2. Instead of using multi-sorted parallel substitutions, Autosubst uses vector substitutions, that bundle substitutions over multiple variable sorts. Since Rocq is more tactic-oriented, Autosubst provides a tactic to apply these laws, whereas in Agda we opted to embed the laws natively via the rewrite mechanism to achieve definitional equalities. Additionally, Autosubst supports code generation of syntax-dependent functions and laws from a Higher-Order Abstract Syntax (HOAS) definition, while we currently have no such counterpart. If we add code generation in the future we prefer a solution that uses reflection, to generate correct code within Agda instead of relying on external code generation. Unlike our approach, Autosubst does not provide an abstraction similar to Kits for unifying proofs about renamings and substitutions.

The idea of Kits was originally proposed by McBride in an intrinsically typed setting, which is also reflected new work, for instance in [4]. The latter also experiments with a uniform treatment of renamings and substitutions in an

[5] The ‘X constructor allows the addition of S' many binders. In its current state, Agdasubst does not support variadic binders, the rewrite rules only apply when S' is fixed.

intrinsically typed setting. Saffrich [18] developed a framework based on the Kit concept for scoped syntax, but without providing any strategy for automatically solving equations between terms with substitutions. Their focus was instead on abstracting over typing relations and the semantics of the object language, while providing a uniform treatment of renamings and substitutions in that context. These results apply directly to Agdasubst, as demonstrated in Section 5.1, where we use the typing Kits from their work. Our laws integrate seamlessly with these extension Kits and substantially simplify the proofs required for their instantiation.

There already exists another Agda library with a similar goal to Agdasubst, called Abstract Binding Trees, which is, for example, used by Siek and Chen [21]. It allows one to construct an object language from a generic syntax, similar to the one in Section 5.2, but in contrast it supports only extrinsic scoping, not intrinsic. Rather than focusing on specific relations, for instance the typing relation, as in [18], it abstracts over arbitrary predicates about the language. The library also offers experimental support for rewriting, but the current state of this feature is unknown to us. In any case, rewriting is only supported in the style of treating renamings and substitutions separately, as in Section 3.2, rather than in a uniform threatement.

More distantly related is Nominal 2 [24], an Isabelle package that addresses the α-equivalence problem not via de Bruijn indice, but using nominal datatypes and freshness conditions. This approach is closer to the intuitive notion of variables, but the tool does not provide any strategy for solving equations involving terms with substitutions.

Additionally, there are tools such as Needle and Knot [14], which is a code generator for unscoped syntax using de Bruijn indices. This tool generate substitution and interaction lemmas for single-pointed substitutions in languages that feature multiple variable sorts and binders capable of binding lists of variables.

7 Further Work

This section outlines possible directions for improving Agdasubst in the future.

Reflection

In comparison to other libraries discussed in Section 6, users of Agdasubst still need to manually provide the definitions for traversal, the syntax-dependent proofs (e.g. right-id and compositionality). Saffrich has shown that these definitions can be derived automatically using Agda's reflection mechanism. There is no reason why the same approach could not be applied for Agdasubst.

Sigma Calculus Extensions

Stark introduced several interesting extensions to the σ-calculus that could also be added to Agdasubst. One example is support for variadic binders, that is, binders of a variable or unknown number of variables, which are for instance useful for languages with pattern matching. Another possible extension is support for lists of terms, allowing a variable or unknown number of terms in the syntax type. This would, for instance, enable support for languages with n-ary application. Both of these features can be encoded in the current system by inlining the list data structure constructors directly into the syntax definition, treating them as terms of a new sort.

Meta Theory

In its current state, Agdasubst completely lacks a meta-theoretical analysis. While we conjecture it to be terminating and confluent, it would be interesting to formally establish these properties.

Completeness

As mentioned, Agdasubst is not complete once we move beyond reasoning about only renamings or substitutions. As soon as we cross the boundary between these two worlds, by considering more general maps or Kits, we definitely cannot expect Agdasubt to solve all possible equations. It would therefore be worthwhile to investigate additional laws, similar in spirit to the coincidence laws, to increase the space of solvable equations.

Performance

At the moment, the performance of Agdasubst is, to be honest, very poor. For instance, the proof of subject reduction for System F with subtyping requires several minutes to check. Our analysis shows that most of this time is spent on instance resolution and function coverage analysis (i.e., verifying that pattern matching clauses are exhaustive). Currently, we have no concrete strategy to improve these aspects, without decreasing user-experience, except that we know, that the addition of the coincidence laws dramatically increases type-checking time.

8 Conclusion

We have shown how to take advantage of the laws of the σ-calculus to automatically solve equations inside proofs about syntax with binders, in Agda. We first embedded the σ-calculus with first-class renamings directly into Agda as reduction rules, using its rewrite mechanism to yield definitional equalities.

We then wanted to unify proofs about renamings and substitutions, which led to the current state of the Agdasubst library. To achieve our goal, we introduced the notions of Kits and Maps from prior work and adapted the σ-calculus laws to this setting. We demonstrated how Agdasubst can be applied to establish properties of formal languages, such as subject reduction for System F with subtyping. The Agdasubst library is still far away from a stable release candidate. We have outlined some directions for future development and plan to improve the library in the time to come.

References

[1] M. Abadi, L. Cardelli, P.-L. Curien, and J.-J. Levy. 1989. Explicit substitutions. In *Proceedings of the 17th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (San Francisco, California, USA) *(POPL '90)*. Association for Computing Machinery, New York, NY, USA, 31–46. doi:10.1145/96709.96712

[2] Robin Adams. 2006. Formalized Metatheory with Terms Represented by an Indexed Family of Types. In *Types for Proofs and Programs*, Jean-Christophe Filliâtre, Christine Paulin-Mohring, and Benjamin Werner (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 1--16.

[3] Guillaume Allais, Robert Atkey, James Chapman, Conor McBride, and James McKinna. 2021. A Type and Scope Safe Universe of Syntaxes with Binding: Their Semantics and Proofs. arXiv:2001.11001 [cs.PL] https://arxiv.org/abs/2001.11001

[4] Thorsten Altenkirch, Nathaniel Burke, and Philip Wadler. 2025. Substitution without copy and paste. https://nottingham-repository.worktribe.com/output/51884690/substitution-without-copy-and-paste. In *Logical Frameworks and Meta-Languages: Theory and Practice (LFMTP 25)* (2025-07-19). Birmingham, UK.

[5] Nick Benton, Chung-Kil Hur, Andrew Kennedy, and Conor McBride. 2012. Strongly Typed Term Representations in Coq. *Journal of Automated Reasoning - JAR* 49 (08 2012), 1--19. doi:10.1007/s10817-011-9219-0

[6] L. Cardelli, S. Martini, J.C. Mitchell, and A. Scedrov. 1994. An Extension of System F with Subtyping. *Information and Computation* 109, 1 (1994), 4--56. doi:10.1006/inco.1994.1013

[7] Jesper Cockx. 2020. Type Theory Unchained: Extending Agda with Userined Rewrite Rules. In *25th International Conference on Types for Proofs and Programs (TYPES 2019) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 175)*, Marc Bezem and Assia Mahboubi (Eds.). Schloss Dagstuhl -- Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 2:1--2:27. doi:10.4230/LIPIcs.TYPES.2019.2

[8] Jesper Cockx and Andreas Abel. 2018. Elaborating dependent (co)pattern matching. *Proc. ACM Program. Lang.* 2, ICFP, Article 75 (July 2018), 30 pages. doi:10.1145/3236770

[9] Jesper Cockx, Nicolas Tabareau, and Théo Winterhalter. 2021. The taming of the rew: a type theory with computational assumptions. *Proc. ACM Program. Lang.* 5, POPL, Article 60 (Jan. 2021), 29 pages. doi:10.1145/3434341

[10] Pierre-Louis Curien, Thérèse Hardin, and Jean-Jacques Lévy. 1996. Confluence properties of weak and strong calculi of explicit substitutions. *J. ACM* 43, 2 (March 1996), 362–397. doi:10.1145/226643.226675

[11] Jean-Yves Girard. 1972. *Interprétation fonctionnelle et élimination des coupures de l'arithmétique d'ordre supérieur.* PhD thesis. Éditeur inconnu.

[12] Healfdene Goguen and James McKinna. 1997. Candidates for substitution. *LFCS report series-Laboratory for Foundations of Computer Science ECS LFCS* (1997).

[13] Daniel Gratzer, Jonathan Sterling, Carlo Angiuli, Thierry Coquand, and Lars Birkedal. 2022. Controlling unfolding in type theory. *arXiv preprint arXiv:2210.05420* (2022).

[14] Steven Keuchel, Stephanie Weirich, and Tom Schrijvers. 2016. Needle & Knot: Binder Boilerplate Tied Up. In *Proceedings of the 25th European Symposium on Programming Languages and Systems - Volume 9632*. Springer-Verlag, Berlin, Heidelberg, 419–445. doi:10.1007/978-3-662-49498-1_17

[15] Per Martin-Löf. 1975. An Intuitionistic Theory of Types: Predicative Part. In *Logic Colloquium '73*, H.E. Rose and J.C. Shepherdson (Eds.). Studies in Logic and the Foundations of Mathematics, Vol. 80. Elsevier, 73--118. doi:10.1016/S0049-237X(08)71945-1

[16] Conor McBride. 2005. Type-preserving renaming and substitution. (2005). http://strictlypositive.org/ren-sub.pdf Unpublished manuscript.

[17] Ulf Norell. 2007. *Towards a Practical Programming Language Based on Dependent Type Theory.* PhD Thesis. Department of Computer Science and Engineering, Chalmers University of Technology, SE-412 96 Göteborg, Sweden. https://wcl.cs.rpi.edu/pilots/library/papers/TAGGED/4230-Norell(2007)-Towardsapracticalprogramminglanguagebasedondependenttypetheory.pdf

[18] Hannes Saffrich. 2024. Abstractions for Multi-Sorted Substitutions. In *15th International Conference on Interactive Theorem Proving (ITP 2024) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 309)*, Yves Bertot, Temur Kutsia, and Michael Norrish (Eds.). Schloss Dagstuhl -- Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 32:1--32:19. doi:10.4230/LIPIcs.ITP.2024.32

[19] Steven Schäfer, Gert Smolka, and Tobias Tebbi. 2015. Completeness and Decidability of de Bruijn Substitution Algebra in Coq. In *Proceedings of the 2015 Conference on Certified Programs and Proofs* (Mumbai, India) *(CPP '15)*. Association for Computing Machinery, New York, NY, USA, 67–73. doi:10.1145/2676724.2693163

[20] Steven Schäfer, Tobias Tebbi, and Gert Smolka. 2015. Autosubst: Reasoning with de Bruijn terms and parallel substitutions. In *International Conference on Interactive Theorem Proving*. Springer, 359--374.

[21] Jeremy G Siek and Tianyu Chen. 2021. Parameterized cast calculi and reusable meta-theory for gradually typed lambda calculi. *Journal of Functional Programming* 31 (2021), e30.

[22] Kathrin Stark. 2020. *Mechanising Syntax with Binders in Coq.* Ph. D. Dissertation. Saarland University, Saarbrücken, Germany. Dissertation zur Erlangung des Grades des Doktors der Ingenieurwissenschaften (Dr.-Ing.).

[23] Kathrin Stark, Steven Schäfer, and Jonas Kaiser. 2019. Autosubst 2: reasoning with multi-sorted de Bruijn terms and vector substitutions *(CPP 2019)*. Association for Computing Machinery, New York, NY, USA, 166–180. doi:10.1145/3293880.3294101

[24] Christian Urban, Stefan Berghofer, and Cezary Kaliszyk. 2013. Nominal 2. *Archive of Formal Proofs* (February 2013). https://isa-afp.org/entries/Nominal2.html, Formal proof development.

[25] P. Wadler and S. Blott. 1989. How to make ad-hoc polymorphism less ad hoc. In *Proceedings of the 16th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (Austin, Texas, USA) *(POPL '89)*. Association for Computing Machinery, New York, NY, USA, 60–76. doi:10.1145/75277.75283